ORIGINAL ARTICLE

Decentralized federated learning over noisy labels: a majority voting method

Guan Huang · **Tao Shu**

Received: 29 January 2026 / Accepted: 16 September 2026
© The Author(s), under exclusive licence to Springer-Verlag London Ltd., part of Springer Nature 2026

Abstract
This paper introduces *majority voting based decentralized federated learning* (DFLMV), a general DFL framework for learning from noisy data without relying on any assumptions about local client noise models while maintaining data privacy for all clients. In contrast to conventional majority voting methods that are based on voting on data, a critical novelty of the proposed voting mechanism is to vote on models. Specifically, (1) Clients first perform traditional DFL training on their original local datasets for a predetermined number of communication rounds. (2) Clients use each of their neighbors' models to make a prediction of every data point in their training datasets, then correct the labels based on majority voting. (3) Clients further fine-tune their models based on their updated training dataset. Theoretically we are able to prove that such a vote-on-model method has the benefit of always being able to improve model accuracy after the voting under any noisy label rate, while the conventional voting-on-data methods do not. Extensive experiments conducted on MNIST, Fashion-MNIST, CIFAR-10, CIFAR-10N, CIFAR-100N, Clothing1M, and ANIMAL-10N validate the effectiveness of our proposed approach at various noise levels and different data settings in mitigating the adverse effects of noisy labels.

Keywords Decentralized federated learning · Distributed learning · Federated learning · Majority voting · Label-noise learning

1 Introduction

Labeling data is a fundamental part of preparing datasets for training deep neural networks (DNNs), since models can only learn effectively once raw data are associated with meaningful annotations. Although accurate labels are critical for achieving strong performance [7], some degree of label noise is almost unavoidable in real-world settings, especially when annotations are collected at scale. In practice, labeling is largely carried out by human annotators for tasks such as identifying objects in images, assigning emotions to text, or transcribing speech [41]. Because this process depends heavily on human judgment and sustained attention, annotation quality can be affected by misinterpretations of the task, lapses in concentration, or insufficient domain-specific expertise (e.g., fine-grained classification in datasets such as CUB-200 requires expert-level knowledge in ornithology [47]). Empirically, noisy labels are a common issue across many widely used benchmarks. For instance, a study

Published online: 27 October 2026

conducted by MIT found that about 10% of the QuickDraw dataset, 5.83% of the ImageNet test set, and 5.85% of the CIFAR-100 test set were incorrectly labeled, largely due to annotator carelessness and limited expertise [14].

Recent studies have shown that noisy or low-quality labels can negatively impact multiple aspects of model training, including the trained model's generalization, robustness, interpretability, and overall accuracy, ultimately leading to degraded model performance [7]. This negative impact is even exacerbated under a federated learning (FL) setting, where the model is trained in a distributed way over datasets owned by different clients. As different clients have varying annotation skills, knowledge levels, and attention to detail, some clients' datasets have high-quality labels, while others' do not. Such an unbalanced label quality across different local datasets leads to local models of different qualities, and hence undermining the quality of the global model. Therefore, how to minimize the detrimental effects of noisy labels, which may be unintentionally generated by workers due to their lack of knowledge or carelessness, so as to retain high-quality training over distributed datasets of diverse label qualities remains a critical challenge for practical FL implementation.

Learning with noisy labels has been extensively studied under both centralized learning (CL) and centralized parameter server-based FL (CFL) [27] settings. At a high level, the methods under CL settings can be divided into three different types: (1) loss correction methods [10, 43], (2) clean data preselection methods [5, 30], and (3) noisy label correction methods [39]. The methods under CFL settings can be classified into two categories: (1) noisy label correction methods [50, 55], and (2) noisy label filtering methods [24, 52]. However, existing methods are fundamentally limited by their dependence on a powerful central server (many/one-to-one), making them incompatible with decentralized federated learning (DFL) [19], which does not have a central server but instead relies entirely on peer-to-peer communication (many-to-many) among resource-constrained edge devices (e.g.,connected and automated vehicles (CAVs) and unmanned aerial vehicle constellations (UAVs) [54]). More specifically, the un-applicability of existing methods on DFL is due to the following four main reasons: (1) some methods violate the privacy requirements of DFL. For instance, Englesson and Azizpour [10] require all clients' data samples must be accessible directly by the server in order for it to learn the noise transition matrix. However, in DFL, each client must keep their data local. (2) methods in [23, 29, 30], involve intensive computations under a peer-to-peer setting to select clean labels during training, resulting in high synchronization costs and computation overhead when clients conduct model aggregation. (3) some methods, such as those in [50, 55], rely on stable server-client connections to allow the server to gather information from all clients in real-time for label correction. However, such connections are often impractical in DFL settings, where devices typically operate under loose network conditions with restricted client-to-client communication. (4) methods in [9, 24] require a clean supplementary dataset. However, such a clean dataset is nearly impossible to obtain for DFL, as a client cannot infer clean data for other clients.

In this paper, we focus on learning with noisy labels under the DFL framework, as noisy data presents a more acute problem for this framework due to the lack of a centralized entity to orchestrate the noisy label correction and mitigation process. We expect this work to generate an impact on improving the reliability and accuracy of DFL applications in vital domains such as autonomous-driving vehicles, healthcare, and LEO (Low Earth Orbit) satellites [54].

To mitigate noisy labels in DFL, we propose a three-stage label correction algorithm called DFLMV (*Majority voting based decentralized federated learning*). Specifically, in Stage 1, all clients use traditional DFL to train their local models based on their original local datasets. After completing the predetermined communication rounds assigned to Stage 1, clients proceed to Stage 2, where each client exchanges model parameters with its available communication peers and uses each received peer model to infer a label for every sample in its local training dataset. Among all inferred labels of the same data point, using majority voting, the client picks the most common one and uses it as the updated label of the data. In Stage 3, based on their updated dataset, each client runs extra training epochs to fine-tune its local model obtained from Stage 1. It is also important to note that this paper addresses the commonly seen non-malicious scenario where label errors arise unintentionally due to

annotators' lack of knowledge or recklessness. The malicious attack scenarios, whereby workers/clients collude to inject deliberately fabricated false data and labels, is beyond the scope of this study.

At first glance, the majority voting mechanism proposed above may seem trivial/standard because people tend to believe that majority voting naturally always leads to improvement. This belief is false: majority voting does not always improve the results if the voting is conducted in a straightforward way. For example, consider the following (counter) example, whereby 3 voters are attempting to decide the latent label, say y, of a data point, say x, that is possessed by the dataset of every voter, by exchanging their respective initial belief of x's label, denoted by y_i for voter i, where $1 \leq i \leq 3$, and then conducting majority voting on y. To simplify our presentation but without loss of generality, let us just consider a binary classification scenario, where y and all the y_i's are binary, and the noisy rate of each voter's initial label y_i (i.e., the probability that y_i is actually false) are IID, denoted by ϵ. Based on majority voting, the probability that the voting outcome y is the correct label is simply calculated as the probability that at least 2 voters' initial labels are true. When $\epsilon = 0.5$, it turns out that the voting outcome y has a false label rate of 0.5, exactly the same as the noisy label rate before the voting. It is easy for one to verify that y has an even higher false label rate than ϵ when $\epsilon > 0.5$. In this case, majority voting not only does not help but instead is hurting the latent label accuracy.

Instead of conducting majority voting based on the straightforward exchange of individual voters' (data, label) pairs, a critical novelty in our proposed method is to conduct the voting based on the exchange of individual voters' local model. As will be clear shortly, we are able to prove that majority voting conducted in such a manner always leads to better latent label accuracy under any ϵ. When the noisy label rate of each voter's dataset is non-IID, the same argument is most likely also true based on our extensive experiments, even though we cannot mathematically prove it under the non-IID noisy label condition. This argument is in line with the well-observed fact that learning models often exhibit good robustness to and generalize well over noisy labels, i.e., the trained model typically has higher accuracy than that of the data used for the training [11, 32, 33, 44]. Our proposed method can further boost the accuracy of the model by up to tens of percentage points.

The **main contributions** of this work are summarized as follows:

- A novel vote-on-model majority-voting-based DFL method, DFLMV, is proposed to enable high-quality learning over distributed and noisy-labeled data. In contrast to existing methods, DFLMV has the unique benefits of low computation and communication overhead (as analyzed in Sect. 4.4), preserving local data privacy, and not requiring supplementary clean datasets.
- We establish two key theoretical performance bounds for DFLMV. Firstly, we derive a general upper bound on the generalization error of any DFL algorithm using cross-entropy loss under arbitrary label noise. Secondly, we derive an upper bound on the error rate of majority voting for a multi-class classification problem. Based on these bounds, we rigorously analyze several factors influencing the error rate of our label correction mechanism and prove that DFLMV guarantees a gain over vanilla DFL (i.e., without MV). To make the theoretical proof of the error rate upper bound mathematically tractable, we assume non-colluding neighbors with identical vote distributions in our proof. To evaluate how effective the proposed DFLMV method can perform in real-world environment, we relax the above assumptions and conduct extensive experiments over seven different datasets under various non-IID data/noise settings. As detailed in Sect. 5, DFLMV achieves substantial accuracy gains, improving test accuracy by more than 8.92%, particularly in non-IID settings.
- We conduct extensive experiments on three synthetic datasets (MNIST, Fashion-MNIST, and CIFAR-10) across 12 settings, considering combinations of IID/non-IID data, IID/non-IID noisy labels, and three noise models (symmetric, pairflip, and asymmetric). Additionally, we test DFLMV on four real-world noisy datasets (CIFAR-10N, CIFAR-100N, Clothing1M, and ANIMAL-10N) under non-IID conditions. These experiments verify that the proposed DFLMV approach effectively mitigates the detrimental effects of noisy labels and significantly improves the learned model's accuracy.

Note that even though DFLMV is presented in the context of DFL in this paper, it is also easy to see that the method can be extended to CFL with minor changes, as elaborated in Appendix A.

2 Related work

2.1 Decentralized federated learning

DFL is an emerging FL framework. With DFL, there is no central server for aggregating model parameters. Clients train their models by exchanging their model parameters with each other without divulging any of their local data during the training process. The concept of DFL was first proposed in [21]. In recent years, the DFL structure comes in a wide variety of variants, including sequential pointing line structures [4, 35, 36], cycle pointing ring structures [16, 53], fully connected peer (mesh) structures [3, 6, 34], hybrid structures [37, 42], etc. The primary assumption behind these studies is that every client's local dataset is noise-free.

However, it has been shown that this assumption cannot be held in a practical DFL system because clients have varying annotation skills and personal preferences [7]. As noisy labels are inevitable in decentralized data ownership, it is imperative to consider the existence of noisy labels and work on developing an appropriate method to deal with these noisy labels effectively.

2.2 Learning with label noise

Incomplete patterns and cognitive errors can cause label noise. Learning with noisy labels has been extensively explored in CL and CFL settings. Generally speaking, there are three categories of CL methods: (1) loss correction methods [10, 13, 43]: These methods often assume noisy labels deteriorate from ground-truth labels due to an unknown noise transition matrix T, and these approaches acquiesce to all clients' data participating in model training to learn this matrix T. (2) clean data preselection methods [5, 30]: These methods need to perform computation-intensive procedures to select clean data with several cross-validation iterations during training. (3) noisy label correction methods [26, 39, 40, 47]: These approaches typically require an additional clean dataset for detecting and relabeling noisy labels.

Methods in CFL settings can be roughly classified into two categories: (1) label correction methods [50, 55]: Most of these methods require each client to perform intensive computations during each communication round to generate auxiliary information from their local datasets. This information, along with model parameters, is then exchanged with the server in real-time to train an auxiliary module for future label correction, relying on stable server-client connections. (2) noisy label filtering methods [24, 52]: These approaches typically require a clean supplementary dataset to train the noisy label filter.

However, the above methods cannot be directly applied to the DFL framework due to four main reasons. (1) Methods requiring excessive computational power can lead to high synchronization costs during the aggregation of clients' models. This increases communication overhead for each client's local model convergence, making it difficult to achieve efficient convergence. (2) Methods relying on stable server-client connections for real-time information exchange cannot perform effectively in DFL settings, where devices typically operate under loose network connections and limited client-to-client communication. These limitations hinder each client's ability to access comprehensive, real-time auxiliary information from its neighbors for effective label correction. (3) Methods of acquiescing to all client data participating in model training will violate DFL's privacy policy. (4) Some methods require a clean supplementary dataset. However, such an auxiliary clean dataset is hard to obtain, as noisy labels will be unavoidable in decentralized data ownership. Therefore, it is crucial to develop a practical DFL method that can reduce the negative impact of corrupt labels and ensure high-quality training over diverse datasets with noisy labels.

3 Problem statement

Given a DFL system with $|K|$ clients, where each client $k \in K$ possesses a noisy local training data set $D_k = \{(x_k(i), y_k(i))\}_{i=1}^{|D_k|}$ (with x being the feature and y being the label), our goal is to let each client k construct an improved dataset $\widetilde{D_k}$, containing less noisy data than D_k. Following this, each client aims to find the optimal solution for minimizing the following empirical risk function:

$$\text{argmin}_{w_k \in \mathcal{H}} F\left(w_k, \widetilde{D_k}\right) = \frac{1}{|\widetilde{D_k}|} \sum_{i=1}^{|\widetilde{D_k}|} \mathcal{L}_k\left(g(x_k(i), w_k), y_k(i)\right), \tag{1}$$

where we define

Model Parameter Space: $\mathscr{H} \subseteq \mathbb{R}^h$ denotes the parameter space for a learning model, where $h \in \mathbb{N}$ stands as the dimension of the parameter space. The local model of each client $k \in K$ has the parameter $w_k \in \mathscr{H}$.

Local Dataset: We assume a horizontally partitioned dataset, where the global dataset D is distributed across $|K|$ clients. Each client k possesses a local dataset D_k, containing data points sampled from the global dataset according to a distribution ψ_k.

Noisy Dataset: For each client k, we define the ground truth data distribution as τ_k and the potentially noisy data distribution as ρ_k. Then, a noisy dataset satisfies the following:

$$\Pr_{\tau_k}\left(y \middle| x\right) \neq \Pr_{\rho_k}\left(y \middle| x\right), \tag{2}$$

where Pr refers to the probability function for a given distribution and an event. Therefore, we can simply think that the testing dataset is sampled via τ_k, and the training dataset is sampled via ρ_k.

Metamodel: We define the metamodel as a function $g : \mathbb{R}^{d_x} \times w_k \to [\mathbb{0}, \mathbb{1}]^{d_y}$, where d_x and d_y are the dimensions of the feature space and label space, respectively. This function describes a trained model that predicts labels for given data features and model parameters. For convenience, we have $g(x_k(i), w_k) = \widehat{y_k(i)}$.

Loss Function: The loss function can be described as $\mathcal{L} : [\mathbb{0}, \mathbb{1}]^{d_y} \times \mathbb{R}^{d_y} \to \mathbb{R} \geq 0$. For example, in our case, we use the cross-entropy loss for each client k:

$$\mathcal{L}_k : \left(v_k(i), \widehat{v_k(i)}\right) \to -\sum_{j=1}^{|C|} v_k(i)(j) \cdot \log\left(\widehat{v_k(i)(j)}\right), \tag{3}$$

where C is the set of classes, $v_k(i)(j)$ is the one-hot probability vector, $v_k(i)$ represents the observed value $y_k(i)$, and j represents the jth value in vector $v_k(i)$. $\widehat{v_k(i)(j)}$ is the predicted probability vector, given by: $\widehat{v_k(i)(j)} = \text{Softmax}\left(f_k\left(x_k(i)\right)\right)$, where f_k denotes the raw output (logits) of the model before being processed by the softmax function for client k.

In contrast to our approach, which explicitly addresses noise by constructing $\widetilde{D_k}$, clients in standard DFL directly minimize the empirical risk function on their raw, potentially noisy dataset D_k, which can be represented in the following:

$$\text{argmin}_{w_k \in \mathcal{H}} F\left(w_k,\ D_k\right) = \frac{1}{|D_k|} \sum_{i=1}^{|D_k|} \mathcal{L}_k\left(g(x_k(i), w_k), y_k(i)\right), \tag{4}$$

4 Proposed algorithm : DFLMV

DFLMV is a three-stage DFL training method developed to tackle learning from commonly seen non-malicious label errors, unintentionally generated by workers due to their diverse annotation expertise and carefulness, in datasets owned by distributed entities. DFLMV comprises three stages: initial training stage, label correction stage, and retraining stage, as elaborated below. The pseudocodes of DFL and DFLMV are provided in Algorithm 1 and Algorithm 2, respectively.

Input: Learning rate η, number of global communication rounds E_G, number of local epochs E_L, number of communication peers n_{Peers}, set of clients K
Each Client Executes:
Initialize model weights: w_k^0
$T \leftarrow 0$
while $T < E_G$ **do**
    Randomly select n_{Peers} available clients as communication peers
    Broadcast model parameters w_k^T to the selected peers
    Receive model parameters $\{w_j^T\}_{j=1}^{n_{\text{Peers}}}$ from the selected peers
    Merge the received models: $w_k^T \leftarrow \sum_{j=1}^{n_{\text{Peers}}} \frac{1}{n_{\text{Peers}}} w_j^T$
    Starting from w_k^T, perform E_L local training epochs on D_k with learning rate η_T to obtain w_k^{T+1}
    $T \leftarrow T + 1$
end while

Algorithm 1 Decentralized Federated Learning (DFL)

4.1 Initial training stage

DFLMV begins with the traditional DFL, in which each client will use stochastic gradient descent (SGD) on their local dataset D_k for E_L local epochs to minimize empirical risk function and thus minimize their local training loss. Specifically, each client first needs to get an initial model by doing the gradient descent:

$$w_k^{T+1} \leftarrow w_k^T - \eta_T \nabla F\left(w_k^T, D_{km(T)}\right), \tag{5}$$

where η is the learning rate, and $D_{km(T)}$ stands for the kth client's mini-batch in the Tth epoch.

Once the model has been initialized, each client k broadcasts its parameters w_k to its neighboring clients. Afterward, the client k waits for n_{Peers} model parameters to be received, and once it receives the n_{Peers} model parameters, it aggregates the models by using the FedAvg algorithm:

$$w_k^T \leftarrow \sum_{j=1}^{n_{\text{Peers}}} \frac{1}{n_{\text{Peers}}} w_j^T. \tag{6}$$

The new aggregated model w_k^T will be trained for E local epochs before it is ready to be broadcast again. Let T_{LC} denote the label-correction round. Each client performs the initial decentralized training procedure for communication rounds $t = 1, \ldots, T_{\text{LC}}$. After all rounds assigned to Stage 1 have been completed, the client proceeds to the one-time label-correction operation in Stage 2.

Input: Learning rate η, number of communication rounds E_G, label-correction round T_{LC}, number of local epochs E_L, number of communication peers n_{Peers}, and set of clients K
Each Client Executes:
Stage 1: Initial Training
Initialize model weights w_k^0
$T \leftarrow 0$
while $T < T_{\mathrm{LC}}$ **do**
  Randomly select n_{Peers} clients from those available in round T as communication peers
  Broadcast model parameters w_k^T to the selected peers
  Call OnReceiveModel(w_i^T)
  $T \leftarrow T + 1$
end while
Stage 2: Label Correction via Majority Voting
Randomly select n_{Peers} available clients as voting peers
Receive their latest model parameters
$B \leftarrow n_{\mathrm{Peers}}$
for $i = 1, 2, \ldots, |D_k|$ **do**
  Each client corrects the label using the majority-voting protocol: $Y_k(i) \leftarrow \arg\max_{z \in C} \sum_{j=1}^{B} \mathbb{1}(\widehat{Y}_j(X_k(i)) = z)$, where B is the number of peer models used by client k during label correction, and $\widehat{Y}_j(X_k(i))$ is the label predicted for the ith data point in D_k using the jth peer model.
end for
Stage 3: Retraining
while $T < E_G$ **do**
  Randomly select the round-specific communication peers
  Exchange and aggregate the available peer models
  Starting from w_k^T, perform E_L local training epochs on $\widetilde{D}_k$ to obtain w_k^{T+1}
  $T \leftarrow T + 1$
end while
Function: OnReceiveModel(w_i^T)
Save the received model w_i^T
while Number of received models $< n_{\mathrm{Peers}}$ **do**
  WAIT for additional selected peer models
end while
Merge the saved models: $w_k^T \leftarrow \sum_{j=1}^{n_{\mathrm{Peers}}} \frac{1}{n_{\mathrm{Peers}}} w_j^T$
Starting from w_k^T, perform E_L local training epochs on D_k with learning rate η_T to obtain w_k^{T+1}
End Function

Algorithm 2 Majority-Voting-Based Decentralized Federated Learning (DFLMV)

4.1.1 Upper bound on the generalization error of DFL

In order to analyze how various noisy training datasets affect the performance of a machine learning model, we consider each data point as a multi-dimensional random variable (RV), denoted by (X, Y), where X represents the feature and Y is the label. Accordingly, a dataset of client k can be represented as a vector of random variables:

$$D_k = \{(X_k(1),\ Y_k(1)), \ldots, (X_k(|D_k|),\ Y_k(|D_k|))\}, \tag{7}$$

where $(X_k(i), Y_k(i))$ is the ith data point in kth client's dataset.

We define client k's empirical risk function (given the potential noisy dataset) as:

$$R_k(w_k) = \mathbb{E}_{D_k \sim \rho_k}\left[\mathcal{L}_k\left(g\left(X_k, w_k\right), Y_k\right)\right], \tag{8}$$

where w_k is the model parameter of client k; $\mathbb{E}(.)$ is the expectation function. Similarly, we define the client k's ground-truth risk function (given a clean dataset) as:

$$R_k^*(w_k) = \mathbb{E}_{D_k \sim \tau_k}\left[\mathcal{L}_k\left(g\left(X_k, w_k\right), Y_k\right)\right]. \tag{9}$$

Then we follow [51] to define client k's generalization error of the given model as:

$$G_k(w_k) = \left|R_k^*(w_k) - R_k(w_k)\right|. \tag{10}$$

Theorem 1 *(Upper bound on the generalization error of a given model). For any DFL algorithm under label noise that uses the cross-entropy function as the loss function, its generalization error is upper bounded by:*

$$G_k(w_k) \leq \Omega \cdot \mathbb{E}_{X_k}\left[\sum_{j=1}^{|C|}\left|\Pr_{\rho_k}\left(Y_k = j \middle| X_k\right) - \Pr_{\tau_k}\left(Y_k = j \middle| X_k\right)\right|\right], \tag{11}$$

where Ω is the upper limit among the elements of the vector f_k, which is the raw output produced by the neural network before being processed by the softmax function for client k.

The proof of Theorem 1 is given in Appendix B.1.

Corollary 1 *(Impact of label noise on traditional DFL). A lower label noise ratio will result in a lower generalization error $G_k(w_k)$ and a better performance of the trained model.*

The proof of Corollary 1 is provided in Appendix B.2.

4.2 Label correction stage

During stage two, each client first exchanges model parameters with its online neighbors and then uses each newly received neighbor's model to predict a label for each piece of data in its training dataset. Without loss of generality, we denote B as the total number of models that client k has received and hence could use during the majority voting process. We let $\widehat{Y_j(X_k(i))}$ be the predicted label for feature $X_k(i)$ made by client k using its jth neighbor's model. Among the B labels made for $X_k(i)$, client k selects the most common one according to majority voting and considers this one as the updated label for $X_k(i)$. Hence, the majority voting protocol can be expressed in the following:

$$\widetilde{Y_k(X_k(i))} = mvf\left(\widehat{Y_1(X_k(i))}, \ldots, \widehat{Y_B(X_k(i))}\right) = \operatorname{argmax}_{z \in C}\sum_{j=1}^{B}\mathbb{1}\left(\widehat{Y_j(X_k(i))} = z\right), \tag{12}$$

where $\widetilde{Y_k(X_k(i))}$ is the updated label for $X_k(i)$ and $mvf(.)$ is the majority voting function that returns the label that receives the most votes among all B neighbors.

Afterward, client k replaces the original label with the updated label for all the data points in its local training dataset, i.e., client k will do the following:

$$Y_k(i) \leftarrow \widetilde{Y_k(X_k(i))} \quad (\forall X_k(i) \in D_k). \tag{13}$$

4.2.1 Upper bound on the error rate of majority voting

Without loss of generality, let us focus our analysis on the majority voting process of the first data point of client 1. Such a treatment allows us to drop the index of the client and the index of data in the analysis and, hence, simplify our presentation. In particular, we denote A as the ground truth classification (i.e., the true label) of the target data point, and we let $\widehat{A_j}$ denote the predicted label for the data point made by the target client by using its jth neighbor's model. Then, the discrepancy between $\widehat{A_j}$ and A can be modeled by the conditional probability distribution $\Pr(\widehat{A_j}|\, A)$. Based on the above simplified notations, we can rewrite the $mvf(.)$ equation as:

$$\widetilde{A} = mvf\left(\widehat{A_1}, \ldots\ \widehat{A_B}\right) = \text{argmax}_{z \in C} \sum_{j=1}^{B} \mathbb{1}(\widehat{A_j} = z), \tag{14}$$

where $\widetilde{A}$ is the updated label.

We denote $p_{u|r}^{(j)}$ as the probability that the jth neighbor's model predicted label $\widehat{A_j} = u$ while $A = r$, where $u, r \in C$, i.e.,

$$p_{u|r}^{(j)} = \Pr\left(\widehat{A_j} = \ u | A = r\right). \tag{15}$$

The error rate of the majority voting is defined as:

$$\mathrm{P_e} = \Pr\left(A \neq \widetilde{A}\right). \tag{16}$$

Theorem 2 ***(Upper bound on*** $\mathrm{P_e}$***).*** *Given that neighbors are not colluding in their training and that the distributions of the votes are identical, the error rate of the majority voting is upper bounded by:*

$$\mathrm{P_e} \leq 2\left(1 - \frac{1}{|C|}\sum_{r=1}^{|C|}\prod_{\substack{u=1 \\ u \neq r}}^{|C|}\left(1 - \sum_{\beta=0}^{\infty} e^{-B(p_{u|r}+p_{r|r})} \times \left(\frac{p_{u|r}}{p_{r|r}}\right)^{\frac{\beta}{2}} \times I_\beta(2B\sqrt{p_{u|r}p_{r|r}})\right)\right), \tag{17}$$

where β is an integer, $I_{\beta(.)}$ is the Bessel function of the first kind of order [28], i.e.,

$$I_\beta(x) = \sum_{t=0}^{\infty} \frac{(-1)^k}{t!(t+\beta)!}\left(\frac{x}{2}\right)^{2t+\beta}. \tag{18}$$

We defer the proof of Theorem 2 to Appendix B.3.

Based on the above theorem, we propose two corollaries regarding the impact of the number of neighbors (i.e., B) and the quality of the neighbor's model (represented by the model's generalization error) on $\mathrm{P_e}$.

Corollary 2 *Given that neighbors are not colluding in their training and that the distributions of the votes are identical, the bound on* $\mathrm{P_e}$ *is monotonically decreasing with B. In an extreme case, when B tends to* $+\infty$, $\mathrm{P_e}$ *tends to 0.*

The proof of Corollary 2 is included in Appendix B.4.

Corollary 3 *Given that neighbors are not colluding in their training and that the distributions of the votes are identical, higher quality of neighbor's model (i.e., smaller generalization error of the model) reduces* $\mathrm{P_e}$.

We defer the proof of Corollary 3 to Appendix B.5.

4.3 Retraining stage

In this stage, each client continues training on its updated dataset, denoted by $\widetilde{D}_k$, using the latest model parameters obtained from Stage 1. Specifically, during each of the remaining $E_G - T_{\text{LC}}$ communication rounds, each client exchanges its model parameters with the selected n_{Peers} peers in the current round and aggregates the received peer models according to Eq. (6). The resulting aggregated model w_k^T is subsequently trained for another E_L local epochs according to Eq. (5) before the next communication round. After completing all remaining communication rounds, each client obtains its final fine-tuned model w_k.

Theorem 3 *Define $G_k(w_k^{D_k \sim \rho_k})$ and $G_k(w_k^{\widetilde{D_k} \sim \tilde{\rho}_k})$ as the generalization error of the models trained by client k over D_k and over $\widetilde{D_k}$, respectively, where $\tilde{\rho}_k$ is the noisy label distribution in $\widetilde{D_k}$. Given that neighbors are not colluding in their training and that the distributions of the votes are identical, we have*

$$G_k\left(w_k^{D_k \sim \rho_k}\right) > G_k\left(w_k^{\widetilde{D_k} \sim \widetilde{\rho_k}}\right). \tag{19}$$

The proof of Theorem 3 is provided in Appendix B.6.

Theorem 3 states that DFLMV can effectively mitigate the adverse effects of corrupted labels and improve the learned model's accuracy. Based on this observation, it is also easy to see that if the DFL training converges over both the raw dataset D_k and the idealized noise-free dataset (as if there is an Oracle that can assign a true label for every data in the dataset), then DFLMV must converge over the updated dataset $\widetilde{D_k}$. While Theorem 3 assumes non-colluding neighbors with identical vote distributions, the same result, e.g., DFLMV improves model accuracy, is also true under non-IID conditions, as will be verified by our extensive experiment in Sect. 5.

4.4 Communication and computation overhead analysis

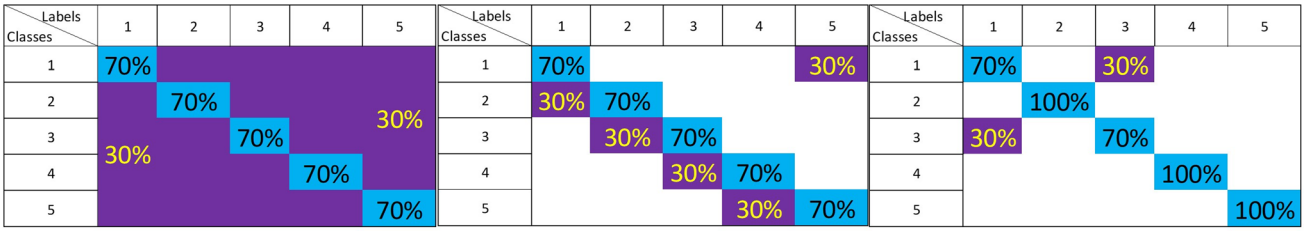
Compared to existing CFL-based label correction methods, which often require additional data processing and training of auxiliary modules for label correction, leading to increased computation and communication overhead, DFLMV offers significant overhead advantages. DFLMV introduces extremely low extra communication overhead over traditional DFL methods. Its overall communication overhead remains $O(m)$, where m is the number of model parameters exchanged among neighbors during the fine-tuning training stage. The extra computation overhead introduced by DFLMV is also very low. Specifically, the only additional computation overhead arises in Stage 2, where the majority voting process for updating labels introduces an $O(n)$ computation overhead, where n is the number of data points. A more detailed communication and computation overhead analysis is available in Appendix B.7.

5 Experiments

In this section, we verify the effectiveness of DFLMV by comparing it with several baseline models across seven datasets under various data/noise settings. Specifically, for each of the synthetic datasets (MNIST [22], Fashion-MNIST [48], and CIFAR-10 [20]), we conduct experiments using a comprehensive set of 12 settings that account for different combinations of IID/non-IID data, IID/non-IID noisy labels, and three different noise models (symmetric, pairflip, and asymmetric). Under each noise model, we consider three different noise ratios: 10%, 30%, and 50%. Additionally, we test DFLMV on four real-world noisy datasets (CIFAR-10N [46], CIFAR-100N [46], Clothing1M [49], and ANIMAL-10N [38]) under non-IID conditions.

Table 1 Summary of datasets and their partitioning in the experiments

Dataset	#Training	#Testing	#Classes	Size of Each Sample
MNIST	60,000	10,000	10	28 × 28 × 1
Fashion-MNIST	60,000	10,000	10	28 × 28 × 1
CIFAR-10	50,000	10,000	10	32 × 32 × 3
CIFAR-10N	50,000	10,000	10	32 × 32 × 3
CIFAR-100N	50,000	10,000	100	32 × 32 × 3
Clothing1M	1,000,000	10,000	14	224 × 224 × 3
ANIMAL-10N	50,000	5,000	10	64 × 64 × 3



(a) Symmetric noise (ε=0.3) (b) Pairflip noise (ε=0.3) (c) Asymmetric noise (ε=0.3)

Fig. 1 Noise models ($[C] = 5$ and the noise ratio $\epsilon = 30\%$ in the illustration). A blue (or purple) grid indicates the fraction of data in the class that has correct (or wrong) labels, respectively.

5.1 Experiment settings

Datasets. We perform extensive experiments on seven benchmark image datasets, including MNIST, Fashion-MNIST, CIFAR-10, CIFAR-10N, CIFAR-100N, Clothing1M, and ANIMAL-10N. The partitioning of training and testing data for these datasets is listed in Table 1.

Generate IID and non-IID datasets. To generate disjoint IID datasets for clients, we independently assign each data sample to a client by following a uniform distribution. For the non-IID case, we focus on label distribution skew [17]. In particular, we partition the original training dataset into $|K|$ disjoint non-IID training datasets via Dirichlet distribution $p_c \sim Dir_K(\alpha)$ [15], where $\alpha \in (0, +\infty)$, and α is the concentration parameter. The smaller the value of α, the greater the level of heterogeneity of the subsets will be. In our case, we chose $\alpha = 1.5$ and $|K| = 10$.

Generate IID and non-IID noisy labels. Due to the heterogeneous nature of clients, the noisy labels can be not only IID but also non-IID among clients' training datasets [12, 50]. We consider three different noisy label models: symmetric noise (Fig. 1 a), pairflip noise (Fig. 1 b), and asymmetric noise (Fig. 1 c). Specifically, to generate IID noisy labels in the symmetric noise model, a fraction of data in each class will flip their labels respectively to a randomly selected (wrong) label. In the pairflip noise model, a fraction of data in each class will flip their labels to the label of the next class. In the asymmetric noise model, a fraction of data in two similar classes will swap their labels. In particular, in the CIFAR-10 dataset, a fraction of data in the automobile class swaps label with that in the truck class (denoted as automobile↔truck) and cat↔dog; in MNIST dataset, we have 1↔7 and 0↔6; in Fashion-MNIST dataset, we have T-shirt↔shirt and pullover↔coat. For each noise model, we consider three noise ratios (i.e., the fraction of data that has wrong labels): 10%, 30%, and 50%. To generate non-IID noisy labels, we first generate IID noisy labels for each client based on the aforementioned process. We then collect the data points of wrong labels from all clients and re-assign these data points to clients based on $Dir_K(\alpha = 1.5)$.

Note that we do not introduce extra label noise to CIFAR-10N, CIFAR-100N, Clothing1M, and ANIMAL-10N, as these datasets already contain real-world label noises. Similar to previous studies [24], we only consider non-IID data partitions for these real-world noisy datasets.

Peer connectivity. To model the limited peer connectivity commonly encountered in decentralized systems, including low Earth orbit satellite constellations, UAV networks, and IoT sensor networks, in each communication

round, each client randomly selects $n_{\text{peers}} = 5$ other clients as its communication peers. Each client can exchange its model only with the selected peers in that round and, when required by a baseline method, also can exchange any additional method-specific information with those peers.

Baselines and model parameters. We compare DFLMV with FedAVG [25] and modified PENS [31]. We modified the settings of PENS so that each client sends and receives model parameters from the same $n_{\text{peers}} = 5$ selected communication peers in order to achieve a fair comparison with DFLMV. Our experiments utilize different hyperparameter settings for various datasets. For MNIST and Fashion-MNIST, we use SGD with zero momentum, weight decay 0.001, learning rate 0.01, and batch size 50. We use $E_G = 300$ communication rounds, with $E_L = 3$ local epochs per communication round. For CIFAR-10, CIFAR-10N, CIFAR-100N, Clothing1M, and ANIMAL-10N, we use SGD with momentum 0.9, weight decay 0.0005, and learning rate 0.001. The corresponding batch sizes are 32, 16, 16, 10, and 16, respectively. We use $E_G = 400$ communication rounds for CIFAR-10, CIFAR-10N, and CIFAR-100N, and $E_G = 200$ communication rounds for Clothing1M and ANIMAL-10N, with $E_L = 3$ local epochs per communication round. Stage 1 is trained for up to T_{LC} communication rounds and is terminated early if the training loss does not decrease for 10 consecutive communication rounds. The one-time majority-voting label correction is then performed in Stage 2. Specifically, we set $T_{\text{LC}} = 150$ for MNIST, $T_{\text{LC}} = 200$ for Fashion-MNIST, CIFAR-10, CIFAR-10N, and CIFAR-100N, and $T_{\text{LC}} = 100$ for Clothing1M and ANIMAL-10N. The dataset-specific values of E_G, T_{LC}, and the remaining Stage 3 training budget are summarized in Table 2. Under each experimental setting, all methods are run five times with random seeds 0, 1, 2, 3, 4. Within each run, all compared methods use the same data partition, label-noise realization, and model initialization seed.

We only use non-pretrained models at the beginning of our experiments. This is because pre-trained models, such as the ResNet families, are trained on large datasets. Using them from the initial step can introduce a potential confound when evaluating the efficacy of our label correction method. Specifically, the improvements observed after label correction could be partially attributed to the pre-existing knowledge embedded in the pre-trained models rather than solely to the correction method itself. Starting with non-pretrained models precludes the impact of the aforementioned bias, allowing a more accurate assessment of the performance gains contributed only by the proposed correction methods. All experiments are executed on Tesla P100 ×16. The details on each network structure are given in Appendix C.

5.2 Main experiment results

Experiment results are given in Tables 3 through 7, where for each learning method, we report the average classification accuracy of all clients' models on the 7 benchmark datasets. After observing these tables, our major insight is that while DFLMV leads to a significant enhancement in the average accuracy of the learning models

Table 2 Training-stage durations used for each dataset

Dataset	MAX total rounds	MAX stage 1 rounds	Stage 3 rounds
	E_G	T_{LC}	$E_G - T_{\text{LC}}$
MNIST	300	150	150
Fashion-MNIST	300	200	100
CIFAR-10	400	200	200
CIFAR-10N	400	200	200
CIFAR-100N	400	200	200
Clothing1M	200	100	100
ANIMAL-10N	200	100	100

E_G denotes the maximum total number of communication rounds, and T_{LC} denotes the maximum number of communication rounds assigned to Stage 1. Stage 2 is triggered either when Stage 1 satisfies the stopping criterion or when $T = T_{\text{LC}}$, whichever occurs first. Specifically, Stage 1 is considered converged if the training loss does not decrease for 10 consecutive communication rounds. T_{LC} is introduced to account for the possible case where Stage 1 does not converge due to a high noisy-label rate. The number of communication rounds assigned to Stage 3 is fixed as $E_G - T_{\text{LC}}$

https://doi.org/10.1007/s00521-026-12582-2

Table 3 Test accuracy results under IID datasets and IID noisy labels setting, reported as mean ± std over five runs

IID Data and IID label-noise distributions											
Dataset	Method	Test accuracy (%)									
		Clean	Symmetric			Pairflip			Asymmetric		
		0	0.1	0.3	0.5	0.1	0.3	0.5	0.1	0.3	0.5
MNIST	FedAvg (FL)	99.14 ± 0.10	98.28 ± 0.24	98.01 ± 0.29	97.65 ± 0.37	98.76 ± 0.21	97.71 ± 0.34	52.82 ± 1.06	98.22 ± 0.18	98.04 ± 0.26	80.96 ± 0.57
	PENS (DFL)	99.43 ± 0.15	98.46 ± 0.27	98.08 ± 0.32	97.31 ± 0.41	98.15 ± 0.25	97.87 ± 0.36	51.46 ± 0.93	98.27 ± 0.22	98.14 ± 0.30	80.89 ± 0.53
	DFLMV (DFL)	99.17 ± 0.09	98.47 ± 0.23	98.19 ± 0.28	97.64 ± 0.35	98.27 ± 0.20	97.89 ± 0.31	60.75 ± 0.73	98.14 ± 0.16	98.22 ± 0.33	84.23 ± 0.46
Fashion-MNIST	FedAvg (FL)	92.60 ± 0.38	89.95 ± 0.48	89.23 ± 0.62	87.34 ± 0.79	90.14 ± 0.52	83.94 ± 0.80	45.79 ± 1.24	90.01 ± 0.45	89.11 ± 0.60	75.65 ± 0.88
	PENS (DFL)	91.64 ± 0.40	89.80 ± 0.45	89.17 ± 0.58	87.47 ± 0.73	89.92 ± 0.49	88.48 ± 0.68	46.03 ± 1.12	90.21 ± 0.42	89.43 ± 0.55	75.18 ± 0.84
	DFLMV (DFL)	91.89 ± 0.39	90.06 ± 0.43	89.15 ± 0.50	87.41 ± 0.66	90.13 ± 0.47	89.80 ± 0.61	50.81 ± 0.97	90.05 ± 0.50	89.46 ± 0.54	75.15 ± 0.77
CIFAR-10	FedAvg (FL)	71.57 ± 0.63	58.69 ± 0.82	55.82 ± 0.95	46.71 ± 1.31	61.39 ± 0.89	45.30 ± 1.42	32.07 ± 1.66	62.93 ± 0.76	59.96 ± 0.91	56.56 ± 1.08
	PENS (DFL)	71.03 ± 0.64	59.18 ± 0.87	55.16 ± 1.03	47.44 ± 1.38	61.43 ± 0.94	45.97 ± 1.49	32.18 ± 1.74	62.97 ± 0.80	60.54 ± 0.99	56.41 ± 1.15
	DFLMV (DFL)	71.27 ± 0.55	58.92 ± 0.78	57.64 ± 0.92	49.72 ± 1.19	61.51 ± 0.85	46.22 ± 1.27	33.07 ± 1.52	64.09 ± 0.71	61.19 ± 0.86	56.75 ± 1.01

over its counterparts across the majority of the tested cases, the magnitude of improvement depends on the level of heterogeneity of the datasets (as measured by the distribution of data and the quality of their labeling) owned by different clients.(Table 4)

More specifically, in cases where clients' datasets contain heterogeneous data and noise, DFLMV typically provides significant model accuracy improvement. These are relevant to the situations where the data is non-IID (Table 5), the noisy label is non-IID (Table 4), both the data and the noisy label are non-IID (Table 6), and the real-world scenarios (Table 7). These are also relevant to the situation where even though both data and noisy labels are IID, the noisy label distributions are statistically different across classes (i.e., the "Pairflip" and "Asymmetric" columns in Table 3). In these cases, it can be observed that accuracy improvement brought by DFLMV is apparent, typically ranging from about several percent to over 20%: the greater the diversity, the more significant the improvement. For example, under a high noisy label ratio of 0.5, Table 6 shows that DFLMV improves test accuracy by 23.68% and 20.95% over PENS and FedAvg, respectively. This observation on the accuracy gain obtained under non-IID scenarios is not surprising because the majority-voting operation is performed after completion of the predetermined Stage 1 training rounds. This design allows each client first to benefit from the vanilla FL learning process, which improves each client's local model accuracy even under non-IID settings (i.e., significant variations in data distribution among clients). More specifically, this initial training ensures that each client's model benefits from shared insights among neighbors while retaining its unique understanding of local data. Consequently, even in the extreme scenario where the training datasets of different clients possess data of different classes, the initial training still helps a client to learn a model that is adapted to the global data distribution (instead of being restricted to the client's local training data). As a result, in the subsequent majority voting, for a given data point, those clients that are making a correct label prediction will point to the same label, while clients making incorrect predictions will likely point to different labels (this is because for the given data point there is only one label to be a correct label but there are many different labels to be wrong label). Therefore, a majority voting mechanism is likely to return the correct label, leading to an improved dataset with fewer labeling errors, and hence higher model accuracy after being trained over this improved dataset during the retraining stage.

On the other hand, in the less-diverse case where both data and noisy labels are IID across datasets and across classes of the same dataset, the improvement achieved by DFLMV is minor, typically ranging from 0 to just a couple percent. This case is relevant to the "symmetric" columns in Table 3. The slight improvement can be explained by noting the fact that even though the distributions of data/noisy-label are IID, their realizations may not be identical – the probability that the same data point is given the same wrong label in two clients' datasets is low. For example, even when the noise ratio is as high as 50%, for the MNIST dataset, the probability that a datapoint is given the same wrong label in two clients' datasets is $(\frac{1}{2})^2 \times (\frac{1}{9})^2 \times 9 = \frac{1}{36}$. As a result, it is unlikely that the same mislabeled data is used for training at two clients. Therefore, in the proposed label correction stage, even if the target client has mislabeled data in its training dataset, there is still a good chance that most of its neighbors' models have been trained on the correct label of the data point, and hence are able to correct the wrong label via majority voting.

Discussion of the gains under non-identical vote distribution. The main reason why high empirical gains remain under a non-identical vote distribution is that, whether the vote distribution is identical or non-identical, each neighbor only has one vote. For those neighbors that cannot predict the correct label, their votes can be distributed across all possible mis-labels, whereas the neighbors that are able to predict the correct label will all vote for the (same) correct label. Therefore, the "dilution" effect of the wrong votes (i.e., votes in favor of mis-labels) remains under the non-identical vote distribution, leading to similar empirical gains under majority voting to those observed in the identical vote distribution case. Note that here we only consider the commonly seen non-malicious scenario where label errors arise randomly and unintentionally due to annotators' lack of knowledge or recklessness. The malicious attack scenarios, whereby workers/clients collude to misclassify an entire class by intentionally injecting deliberately fabricated false data and labels, are beyond the scope of this study.

https://doi.org/10.1007/s00521-026-12582-2

Table 4 Test accuracy results under IID datasets and non-IID noisy labels setting, reported as mean ± std over five runs

IID Data and non-IID label-noise distributions											
Dataset	Method	Test Accuracy (%)									
		Clean	Symmetric			Pairflip			Asymmetric		
		0	0.1	0.3	0.5	0.1	0.3	0.5	0.1	0.3	0.5
MNIST	FedAvg (FL)	99.14 ± 0.10	98.47 ± 0.47	90.72 ± 0.70	84.67 ± 0.95	77.14 ± 0.74	69.88 ± 1.08	49.19 ± 1.28	94.96 ± 0.47	76.60 ± 0.83	67.51 ± 1.07
	PENS (DFL)	99.43 ± 0.15	98.28 ± 0.40	90.83 ± 0.67	88.03 ± 0.96	75.73 ± 0.75	69.83 ± 1.02	53.18 ± 1.25	95.42 ± 0.45	80.41 ± 0.78	66.95 ± 1.10
	DFLMV (DFL)	99.17 ± 0.09	98.05 ± 0.44	96.03 ± 0.62	95.95 ± 1.02	97.84 ± 0.59	91.01 ± 0.86	70.97 ± 1.19	97.37 ± 0.46	90.42 ± 0.77	73.66 ± 1.03
Fashion-MNIST	FedAvg (FL)	92.60 ± 0.38	83.61 ± 0.75	66.61 ± 1.13	57.66 ± 1.43	64.41 ± 0.96	44.89 ± 1.55	38.43 ± 1.94	72.64 ± 0.70	69.41 ± 1.13	54.49 ± 1.43
	PENS (DFL)	91.64 ± 0.40	83.44 ± 0.68	67.04 ± 1.10	57.27 ± 1.45	62.48 ± 0.79	44.84 ± 1.66	36.70 ± 1.71	73.73 ± 0.85	67.38 ± 1.11	56.51 ± 1.32
	DFLMV (DFL)	91.89 ± 0.39	90.67 ± 0.88	82.52 ± 1.24	80.05 ± 1.28	85.59 ± 0.94	69.33 ± 1.45	53.13 ± 1.59	82.16 ± 0.78	74.32 ± 1.05	69.74 ± 1.14
CIFAR-10	FedAvg (FL)	71.57 ± 0.63	59.97 ± 1.22	48.07 ± 1.49	31.12 ± 1.84	51.96 ± 1.37	39.82 ± 1.86	25.19 ± 2.06	53.15 ± 1.38	44.40 ± 1.30	41.98 ± 1.45
	PENS (DFL)	71.03 ± 0.64	60.14 ± 1.18	49.99 ± 1.44	31.21 ± 1.83	52.19 ± 1.22	39.77 ± 1.54	25.24 ± 1.98	53.95 ± 1.30	47.50 ± 1.36	39.21 ± 1.61
	DFLMV (DFL)	71.27 ± 0.55	60.19 ± 1.07	52.57 ± 1.41	33.65 ± 1.60	58.58 ± 1.23	51.03 ± 1.62	33.69 ± 2.04	59.54 ± 1.39	53.36 ± 1.40	51.79 ± 1.59

https://doi.org/10.1007/s00521-026-12582-2

Table 5 Test accuracy results under non-IID datasets and IID noisy labels setting, reported as mean ± std over five runs

Non-IID Data and IID label-noise distributions											
Dataset	Method	Test accuracy (%)									
		Clean	Symmetric			Pairflip			Asymmetric		
		0	0.1	0.3	0.5	0.1	0.3	0.5	0.1	0.3	0.5
MNIST	FedAvg (FL)	99.07 ± 0.34	97.41 ± 0.67	94.45 ± 0.91	93.78 ± 1.26	93.67 ± 0.58	89.09 ± 0.91	54.37 ± 1.54	91.98 ± 0.75	90.27 ± 0.98	76.03 ± 1.11
	PENS (DFL)	98.92 ± 0.30	96.02 ± 0.70	95.59 ± 0.93	95.51 ± 1.38	94.31 ± 0.62	87.73 ± 1.01	56.26 ± 1.46	93.77 ± 0.53	90.22 ± 0.99	76.08 ± 1.08
	DFLMV (DFL)	98.86 ± 0.35	98.22 ± 0.54	95.64 ± 0.86	94.77 ± 1.06	95.98 ± 0.70	95.90 ± 1.03	74.64 ± 1.57	95.82 ± 0.42	95.10 ± 1.02	84.51 ± 1.14
Fashion-MNIST	FedAvg (FL)	91.97 ± 0.52	69.81 ± 1.04	65.80 ± 1.53	62.40 ± 1.60	73.38 ± 1.13	70.71 ± 1.35	41.17 ± 2.16	72.17 ± 0.87	70.95 ± 1.41	64.54 ± 1.68
	PENS (DFL)	91.16 ± 0.74	70.50 ± 1.03	67.42 ± 1.60	64.66 ± 1.67	73.33 ± 1.10	71.81 ± 1.39	44.31 ± 2.02	73.79 ± 0.88	73.32 ± 1.46	66.01 ± 1.82
	DFLMV (DFL)	91.34 ± 0.61	70.16 ± 1.09	70.08 ± 1.42	69.03 ± 1.81	75.82 ± 1.75	75.74 ± 2.05	54.53 ± 2.14	74.42 ± 1.09	73.37 ± 1.55	66.06 ± 1.69
CIFAR-10	FedAvg (FL)	68.75 ± 0.83	41.87 ± 1.36	29.15 ± 2.03	22.55 ± 1.79	41.70 ± 1.24	28.51 ± 1.78	20.82 ± 2.31	52.98 ± 1.36	51.28 ± 1.76	46.40 ± 2.02
	PENS (DFL)	68.51 ± 1.17	45.10 ± 1.62	26.94 ± 1.93	22.50 ± 1.63	45.02 ± 1.42	28.77 ± 1.83	22.49 ± 2.27	52.36 ± 1.30	50.42 ± 1.32	46.35 ± 1.98
	DFLMV (DFL)	68.58 ± 0.81	52.20 ± 1.40	35.85 ± 2.07	24.88 ± 1.76	48.23 ± 1.83	35.14 ± 2.13	23.45 ± 2.52	54.11 ± 1.92	52.40 ± 1.87	48.29 ± 1.93

https://doi.org/10.1007/s00521-026-12582-2

Table 6 Test accuracy results under non-IID datasets and non-IID noisy labels setting, reported as mean ± std over five runs

Non-IID Data and non-IID Label-Noise Distributions											
Dataset	Method	Test Accuracy (%)									
		Clean	Symmetric			Pairflip			Asymmetric		
		0	0.1	0.3	0.5	0.1	0.3	0.5	0.1	0.3	0.5
MNIST	FedAvg (FL)	99.07 ± 0.34	92.20 ± 1.03	77.03 ± 1.29	71.20 ± 1.79	81.83 ± 0.86	74.02 ± 1.59	54.66 ± 2.11	85.83 ± 1.02	75.95 ± 1.26	71.34 ± 1.86
	PENS (DFL)	98.92 ± 0.30	90.95 ± 0.83	74.60 ± 1.45	69.36 ± 1.58	80.11 ± 1.17	70.93 ± 1.38	51.93 ± 2.20	84.67 ± 0.82	75.59 ± 1.52	71.58 ± 1.65
	DFLMV (DFL)	98.86 ± 0.35	97.51 ± 0.92	95.10 ± 1.51	94.92 ± 1.50	96.39 ± 1.08	86.24 ± 1.62	75.61 ± 1.97	95.41 ± 1.07	91.02 ± 1.36	84.21 ± 1.78
Fashion-MNIST	FedAvg (FL)	91.97 ± 0.52	78.66 ± 1.47	67.75 ± 1.77	63.58 ± 2.23	68.70 ± 1.43	51.57 ± 2.37	18.97 ± 2.89	80.24 ± 1.48	71.38 ± 1.76	70.34 ± 2.34
	PENS (DFL)	91.16 ± 0.74	77.71 ± 1.31	68.79 ± 1.93	64.26 ± 2.03	70.18 ± 1.67	51.18 ± 2.16	20.84 ± 3.01	78.98 ± 1.31	73.24 ± 2.06	69.87 ± 2.12
	DFLMV (DFL)	91.34 ± 0.61	83.72 ± 1.40	83.55 ± 1.99	81.96 ± 1.95	82.01 ± 1.50	59.54 ± 2.44	33.17 ± 2.72	90.88 ± 1.55	80.25 ± 1.87	79.56 ± 2.25
CIFAR-10	FedAvg (FL)	68.75 ± 0.83	47.93 ± 1.88	40.31 ± 2.14	29.99 ± 2.70	36.88 ± 1.84	32.62 ± 2.56	16.50 ± 3.13	49.57 ± 1.73	47.62 ± 1.97	43.67 ± 2.69
	PENS (DFL)	68.51 ± 1.17	44.29 ± 1.69	39.29 ± 2.28	28.97 ± 2.49	34.82 ± 2.09	31.96 ± 2.34	17.24 ± 3.21	49.15 ± 1.59	48.06 ± 2.21	42.06 ± 2.41
	DFLMV (DFL)	68.58 ± 0.81	52.21 ± 1.72	47.13 ± 2.35	37.71 ± 2.42	43.89 ± 1.98	37.33 ± 2.60	23.97 ± 2.94	54.19 ± 1.74	50.73 ± 2.04	48.92 ± 2.53

 https://doi.org/10.1007/s00521-026-12582-2

Table 7 Test accuracy results under real-world noisy datasets with non-IID Data setting, reported as mean ± std over five runs

Real-world noisy datasets with non-IID data				
Method	Test accuracy (%)			
	CIFAR-10N (Worst)	CIFAR-100N (Worst)	Clothing1M	ANIMAL-10N
Noise Type	Real-world	Real-world	Real-world	Real-world
Noise Ratio	0.402	0.402	0.38	0.08
FedAvg (FL)	49.47 ± 1.08	30.11 ± 1.34	34.62 ± 0.83	54.58 ± 0.76
PENS (DFL)	50.84 ± 0.94	29.54 ± 1.49	35.49 ± 0.97	55.47 ± 0.88
DFLMV (DFL)	59.92 ± 1.21	33.68 ± 1.12	38.86 ± 1.05	58.16 ± 0.92

Table 8 Average test accuracy (%) under non-IID data and non-IID label-noise distributions at a noise ratio of0.5. Results are reported as mean ± std over five runs.

Non-IID Data and Non-IID Label-Noise Distributions				
Dataset	Method	Test Accuracy (%)		
		Symmetric	Pairflip	Asymmetric
MNIST	FedAvg (FL)	71.20 ± 1.79	54.66 ± 2.11	71.34 ± 1.86
	PENS (DFL)	69.36 ± 1.58	51.93 ± 2.20	71.58 ± 1.65
	FedCorr-DFL	84.63 ± 1.94	67.46 ± 2.48	76.28 ± 1.88
	FedDiv-DFL	86.08 ± 1.83	67.12 ± 2.36	77.19 ± 1.97
	CLC-DFL	81.92 ± 2.14	62.09 ± 2.71	73.64 ± 2.11
	DFLMV (DFL)	94.92 ± 1.50	75.61 ± 1.97	84.21 ± 1.78
Fashion-MNIST	FedAvg (FL)	63.58 ± 2.23	18.97 ± 2.89	70.34 ± 2.34
	PENS (DFL)	64.26 ± 2.03	20.84 ± 3.01	69.87 ± 2.12
	FedCorr-DFL	71.89 ± 2.42	25.31 ± 3.18	73.84 ± 2.36
	FedDiv-DFL	73.04 ± 2.29	26.74 ± 3.03	74.71 ± 2.24
	CLC-DFL	69.86 ± 2.67	24.75 ± 3.41	72.43 ± 2.55
	DFLMV (DFL)	81.96 ± 1.95	33.17 ± 2.72	79.56 ± 2.25
CIFAR-10	FedAvg (FL)	29.99 ± 2.70	16.50 ± 3.13	43.67 ± 2.69
	PENS (DFL)	28.97 ± 2.49	17.24 ± 3.21	42.06 ± 2.41
	FedCorr-DFL	31.28 ± 2.87	18.13 ± 3.43	45.48 ± 2.83
	FedDiv-1DFL	32.14 ± 2.76	19.27 ± 3.25	45.36 ± 2.72
	CLC-DFL	30.61 ± 3.06	16.14 ± 3.61	44.09 ± 3.01
	DFLMV (DFL)	37.71 ± 2.42	23.97 ± 2.94	48.92 ± 2.53

5.3 Comparison with recent noisy-label FL methods

We further compare DFLMV with three representative methods for federated learning with noisy labels: FedCorr [50], FedDiv [24], and CLC [55]. These methods were originally designed for server-based FL and rely on a central server to construct system-wide noise-related statistics, maintain a global correction model or noise filter, or coordinate consensus-based label correction. Therefore, their original implementations cannot be directly executed in the considered server-free DFL system. For comparison under the same server-free decentralized communication constraints, we construct direct DFL-compatible adaptations, denoted as FedCorr-DFL, FedDiv-DFL, and CLC-DFL. In each communication round, every client independently selects $n_{\text{peers}} = 5$ peers and acts as a local coordinator solely for its own training. The client exchanges its model and the method-specific auxiliary information required by the corresponding baseline with these selected peers, performs the original server-side aggregation and coordination calculations using its locally available neighborhood information, and applies the resulting model or statistics only to its own local update. Detailed adaptation procedures are provided in Appendix C.2. Unless otherwise specified, all method-specific hyperparameters follow the values reported in the original papers.

Table 8 reports the results under the most challenging synthetic setting, where both the client data and noisy-label distributions are non-IID and the noise ratio is 0.5. The three adapted methods generally improve upon FedAvg, indicating that their noise-identification, filtering, and consensus-based correction mechanisms retain some effectiveness when constructed from local neighborhood information. Averaged across the nine dataset–noise-type

combinations, FedCorr-DFL, FedDiv-DFL, and CLC-DFL improve upon FedAvg by 6.01%, 6.82%, and 3.92%, respectively.

The improvements of these adaptations nevertheless remain substantially smaller than those of DFLMV. This is because the original FedCorr, FedDiv, and CLC methods rely on system-wide information constructed by a central server, including a globally consistent clean/noisy client partition, an all-client Federated Noise Filter, or a global class-wise consensus. Under the DFL-compatible adaptations, each client must instead approximate the corresponding system-wide quantity using only itself and the five peers selected in the current communication round. Because both the data and noisy-label distributions are non-IID, this small neighborhood may not represent the overall client population. In particular, differences in class composition, class difficulty, and local model accuracy can affect the LID, loss, and confidence statistics used by these methods. Consequently, variations caused by data heterogeneity may be mistaken for evidence of label corruption, reducing the reliability of neighborhood-level client identification, noise filtering, and class-wise consensus and thereby limiting the resulting accuracy gains. In contrast, DFLMV avoids estimating label corruption from cross-client LID, loss, or confidence statistics, whose values can be affected by non-IID class distributions and differences in local model accuracy. Instead, after the initial DFL training stage, each client directly applies its selected peer models to the same local sample and aggregates their predicted labels through majority voting. Because noisy samples are distributed non-IID across clients, the selected peers are exposed to different noise levels and corruption realizations. For a mislabeled sample at the target client, peer models trained predominantly on correctly labeled samples of the relevant class are likely to agree on its true label. By contrast, when peer models are affected by different corrupted samples, their incorrect predictions are generally less concentrated on a single wrong class. Consequently, the correct predictions can form the plurality or majority vote even when some peer models are inaccurate. The client subsequently retrains its model on the corrected local dataset, allowing the sample-level corrections to translate into improved final accuracy. Consequently, DFLMV achieves the highest accuracy in all nine settings and outperforms the strongest adapted baseline by up to 8.92%.

5.4 Sensitivity to the label-correction round

We further investigate the sensitivity of DFLMV to T_{LC}, which determines the maximum number of communication rounds for Stage 1 and the training budget allocated to Stage 3. As described in Sect. 5.1, the same early-stopping criterion is applied for all settings of T_{LC}. A smaller T_{LC} limits the number of initial-training rounds before label correction but allocates more communication rounds to retraining on the corrected local datasets. Conversely, a larger T_{LC} allows more initial decentralized training when early stopping is not triggered, but allocates fewer communication rounds to retraining after label correction. We conduct this analysis on CIFAR-10 under non-IID data and non-IID label-noise distributions, considering symmetric, pairflip, and asymmetric noise at a noise ratio of 0.5. We set $E_G = 400$ and evaluate $T_{\mathrm{LC}} \in \{100, 150, 200, 250, 300\}$, corresponding to $T_{\mathrm{LC}}/E_G \in \{0.250, 0.375, 0.500, 0.625, 0.750\}$.

As shown in Table 9, increasing T_{LC} from 100 to 200 improves the mean test accuracy by 4.85%, 3.29%, and 6.19% under symmetric, pairflip, and asymmetric noise, respectively. However, when T_{LC} is further increased beyond 250, the accuracies all go down. This is not surprising: When T_{LC} is small (i.e., smaller than 200 in the

Table 9 Sensitivity to T_{LC}, the maximum Stage 1 communication round, on CIFAR-10 under non-IID data and non-IID label-noise distributions with a noise ratio of 0.5. Results are reported as mean ± std over five runs

T_{LC}	T_{LC}/E_G	Symmetric	Pairflip	Asymmetric
100	0.250	32.86 ± 2.76	20.68 ± 3.18	42.73 ± 2.73
150	0.375	36.42 ± 2.49	22.84 ± 2.77	47.76 ± 2.64
200	0.500	37.71 ± 2.42	23.97 ± 2.94	48.92 ± 2.53
250	0.625	37.48 ± 2.57	23.86 ± 2.69	48.63 ± 2.68
300	0.750	36.96 ± 2.68	23.02 ± 3.08	48.14 ± 2.79

experiment), Stage 1 may terminate before the peer models have received sufficient training and aggregation. As a result, the peer predictions used for majority voting can be less accurate, reducing the quality of the corrected labels. When T_{LC} is too large (i.e., over 250), the number of steps available for re-training on the corrected dataset in Stage 3 is insufficient, leading to decreased accuracies.

6 Conclusion and future work

In this paper, we attempt to address the problem of learning from datasets with common non-malicious label errors, which are often present in decentralized ownership due to annotators' lack of expertise or carelessness. We focus on mitigating the adverse effects of corrupted labels when implementing DFL systems. To tackle the issue at hand, we propose a novel method, DFLMV, a general DFL framework that enables all clients to collaborate to address inevitable noisy labels on decentralized data ownership. Particularly, with DFLMV, clients can correct their labels efficiently and cost-effectively while maintaining their local data privacy. In Sect. 4, our theoretical analysis rigorously demonstrates that DFLMV is capable of correcting noisy labels with high confidence. In Sect. 5, extensive experiments conducted on 7 benchmark datasets show that our proposed approach is robust against noisy labels and performs well in diverse noise settings and data settings. We note that this paper only considers an unweighted plurality majority voting as the label correction mechanism. There are many other possible voting methods that could be more effective or suitable for different scenarios, such as weighted majority voting and probabilistic voting. In our future work, we plan to explore more sophisticated voting methods to further improve the accuracy of learning from noisy labels. We believe this paper could lead to new directions in handling noisy labels in DFL, especially in improving model robustness against noisy labels in decentralized data ownership.

Appendix A: Apply our proposed method to CFL

To apply DFLMV to CFL, we can adapt its core concepts to the standard FedAVG [25]. Specifically, in CFL, after completing the predetermined communication rounds assigned to Stage 1, the parameter server aggregates the parameters of all local models and broadcasts the latest global and local models to all clients. Each client then uses the latest received models to predict a label for each data point in its local training dataset. Subsequently, each client will update the labels according to the DFLMV majority voting protocol. Except for these two minor changes, the other steps will remain the same as in the standard FedAVG.

Appendix B: Proofs

B.1: Proof of Theorem 1

Proof To prove Theorem 1, we first need to derive the expectation of cross-entropy loss. Given $(X_k(i),\ Y_k(i)) \sim \zeta_k$ and f_k. The cross-entropy loss in Eq. (3) can be rewritten as:

$$\mathcal{L}_k\left(\widehat{v_k(i)}, v_k(i)\right) = -\sum_{j=1}^{|C|} v_k(i)(j) \cdot \log\left(\mathrm{Softmax}\left(f_k\left(x_k(i)\right)\right)\right) \tag{20}$$

$$= -\sum_{j=1}^{|C|} v_k(i)(j) \cdot \log\left(\frac{e^{f_k^j(x_k(i))}}{\sum_{q=1}^{|C|} e^{f_k^q(x_k(i))}}\right), \tag{21}$$

where f_k represents the vector of raw outputs from the neural network, x_k denotes a realization of X_k, and $e \approx 2.78$. The terms j and q correspond to the jth and qth entries of the vector f_k, respectively. Since $v_k(i)(j)$ is the one-hot probability vector, it satisfies the following:

$$v_k(i)(j) = 1 \quad \text{and} \quad v_k(i)(a) = 0 \quad \text{for} \quad a \neq j. \tag{22}$$

Hence, we can further simplify the $\mathcal{L}_k$ by using the Eq. (22):

$$\mathcal{L}_k : (g(X_k, w_k), Y_k) = -\log\left(\frac{e^{f_k^j(X_k)}}{\sum_{q}^{|C|} e^{f_k^q(X_k)}}\right) \tag{23}$$

$$= -\left[f_k^j(X_k) - \log\left(\sum_{q}^{|C|} e^{f_k^q(X_k)}\right)\right]. \tag{24}$$

Thus, by using the conditional expectation formula, the expectation of cross-entropy loss can be written as follows:

$$\mathbb{E}\left[\mathcal{L}_k(g(X_k, w_k), Y_k)\right] = \sum_{j=1}^{|C|} \Pr_{\zeta_k}(Y_k = j)\, \mathbb{E}_{X_k|Y_k=j}\left[\mathcal{L}_k(g(X_k, w_k), Y_k)\right] \tag{25}$$

$$= -\sum_{j=1}^{|C|} \Pr_{\zeta_k}(Y_k = j)\, \mathbb{E}_{X_k|Y_k=j}\left[f_k^j(X_k) - \log\left(\sum_{q}^{|C|} e^{f_k^q(X_k)}\right)\right], \tag{26}$$

By using Eq. (26) and the expansion ideal from [18], we can get:

$$G_k(w_k) = |R_k^*(w_k) - R_k(w_k)| \tag{27}$$

$$= \left|\sum_{j=1}^{|C|}\left[\int_{X_k} f_k^j(x_k)\, d\Pr_{\rho_k}(x_k, y_k) - \int_{X_k} f_k^j(x_k)\, d\Pr_{\tau_k}(x_k, y_k)\right]\right| \tag{28}$$

$$= \left|\sum_{j=1}^{|C|}\left[\int_{X_k} f_k^j(x_k)\left(d\Pr_{\rho_k}(x_k, y_k) - d\Pr_{\tau_k}(x_k, y_k)\right)\right]\right| \tag{29}$$

$$= \left|\mathbb{E}_{X_k}\left[\sum_{j=1}^{|C|} f_k^j(X_k)\left(\Pr_{\rho_k}\left(Y_k = j \middle| X_k\right) - \Pr_{\tau_k}\left(Y_k = j \middle| X_k\right)\right)\right]\right| \tag{30}$$

$$\text{RHS of Eq. (30)} \leq \mathbb{E}_{X_k}\left[\sum_{j=1}^{|C|} f_k^j(X_k)\left|\Pr_{\rho_k}\left(Y_k = j \middle| X_k\right) - \Pr_{\tau_k}\left(Y_k = j \middle| X_k\right)\right|\right] \tag{31}$$

$$\text{RHS of Eq. (31)} \leq \Omega \cdot \mathbb{E}_{X_k}\left[\sum_{j=1}^{|C|}\left|\Pr_{\rho_k}\left(Y_k = j \middle| X_k\right) - \Pr_{\tau_k}\left(Y_k = j \middle| X_k\right)\right|\right]. \tag{32}$$

Hence, we have:

$$G_k(w_k) \leq \Omega \cdot \mathbb{E}_{X_k}\left[\sum_{j=1}^{|C|}\left|\Pr_{\rho_k}\left(Y_k = j \middle| X_k\right) - \Pr_{\tau_k}\left(Y_k = j \middle| X_k\right)\right|\right]. \tag{10}$$

□

B.2: Proof of Corollary 1

Proof To prove Corollary 1, we first need to formulate the similarity between ρ_k *and* τ_k. Let us denote $BC(\rho_k, \tau_k)$ as the Bhattacharyya coefficient [2]. Given $\sum_{i=1}^{|C|}\rho_k(i) = 1$ and $\sum_{i=1}^{|C|}\tau_k(i) = 1$, the similarity between ρ_k and τ_k is measured by the following: □

$$\cos(\theta) = BC(\rho_k, \tau_k) = \sum_{i=1}^{|C|}\sqrt{\rho_k(i)\tau_k(i)}, \tag{33}$$

where θ is the difference between ρ_k *and* τ_k.

From Eq. (10) and Eq. (33), we can easily infer that a lower noisy ratio leads to closer proximity between ρ_k *and* τ_k and a smaller value of $|\Pr_{\rho_k}\left(Y_k = z \middle| X_k\right) - \Pr_{\tau_k}\left(Y_k = z \middle| X_k\right)|$, thereby resulting in a smaller $G_k(w_k)$. For example, in a special case, if ρ_k *and* τ_k are identical, then we have:

$$\cos(\theta) = BC(\rho_k, \tau_k) = \sum_{i=1}^{|C|}\sqrt{\rho_k(i)\tau_k(i)} = \sum_{i=1}^{|C|}\sqrt{\rho_k(i)^2} = 1. \tag{34}$$

Since $\cos(\theta) = 1$, we can get $\theta = 0$. Hence, we have:

$$\left|\Pr_{\rho_k}\left(Y_k = j \middle| X_k\right) - \Pr_{\tau_k}\left(Y_k = j \middle| X_k\right)\right| = \left|\Pr_{\rho_k}\left(Y_k = j \middle| X_k\right) - \Pr_{\rho_k}\left(Y_k = j \middle| X_k\right)\right| = 0. \tag{35}$$

By substituting Eq. (35) into Eq. (10), we can get:

$$G_k(w_k) \leq \Omega \cdot \mathbb{E}_{X_k}\left[\sum_{j=1}^{|C|}(0)\right] = 0. \tag{36}$$

Therefore, when the noisy ratio $\to 0$, we can get $G_k(w_k) \to 0$.

B.3: Proof of Theorem 2

Proof To prove Theorem 2, we first need to derive the conditional probability distribution of the vote count for a given class. We let $\mathrm{S} = (S_1, S_2, S_3, S_4, \ldots, S_B)$ be a random vector, where $S_i \in [1, B]$ represents the count of votes for a particular class. For instance, the number of votes for class (u) is given by:

$$S_u = \sum_{k=1}^{B}\mathbb{1}(\widehat{A_k} = u). \tag{37}$$

Since the vote distributions are identical, the conditional probability $(p_{u|r}^{(j)})$ is independent of the voter j. Thus, for the remainder of the paper, we streamline our notation by discarding the superscript (j) and representing it simply

as $(p_{u|r})$. Given $\widehat{A_j} = u$ and $A = r$, by utilizing the multinomial distribution formula, we can express the conditional probability distribution of $\boldsymbol{S}$ as follows:

$$\Pr\left(\mathrm{S} = \mathrm{s} \middle| A = r\right) = \frac{B!}{\prod_{i=1}^{|C|} S_i!} \prod_{u=1}^{|C|} (p_{u|r})^{S_u}, \tag{38}$$

where $\sum_{j=1}^{|C|} S_j = B$.

From the above Eq. (38), it is easy to see that the RVs $S_1, S_2, \ldots, S_B$ are mutually dependent. This lack of independence makes deriving the error rate extremely difficult. To address this, we adopt the Poisson approximation, inspired by [1, 28]. We define $\widehat{\mathrm{S}} = (\widehat{S_1}, \widehat{S_2}, \ldots, \widehat{S_B})$ as a vector of RVs; each of the RVs $\widehat{S_i} \in \widehat{\mathrm{S}}$ is independent of each other; $\widehat{S_i}$ follows Poisson distribution, $\widehat{S_i} \sim P(\lambda)$, and $\lambda = B \times (p_{u|r})$. Given that $\widehat{A_j} = u$ and $A = r$, the conditional probability distribution of $\widehat{\mathrm{S}}$ can be rewritten as follows:

$$\Pr\left(\widehat{\mathrm{S}} = \hat{\mathrm{s}} \middle| A = r\right) = \prod_{u=1}^{|C|} \Pr\left(\widehat{S_u} = \widehat{s_u} \middle| A = r\right) \tag{39}$$

$$= \prod_{u=1}^{|C|} \frac{e^{-B\times(p_{u|r})} (B \times (p_{u|r}))^{\widehat{S_u}}}{\widehat{S_u}!}. \tag{40}$$

To connect the probability events of S and $\widehat{\mathrm{S}}$, we define $\varepsilon(\mathrm{S})$ as an event whose probability changes monotonically (either increasing or decreasing) based on the number of participants. Similarly, let $\varepsilon\left(\widehat{\mathrm{S}}\right)$ denote the same event applied to the Poisson case. Leveraging Lemma 1 of [1] and Corollary 5.11 of [28], we can establish the following inequality:

$$\Pr\left(\varepsilon\ (\mathrm{S})\right) \leq 2 \Pr\left(\varepsilon\ \left(\widehat{\mathrm{S}}\right)\right). \tag{41}$$

To make our upper bound more convincing, we consider the worst-case (The distribution of A is uniform over its domain).

$$\Pr(A = r) = \frac{1}{|C|},\ \forall r \in\ C. \tag{42}$$

Then we can rewrite the error rate $\mathrm{P_e}$ as:

$$\mathrm{P_e} = \Pr\left(\widetilde{A} \neq A\right) \tag{43}$$

$$= 1 - \frac{1}{|C|} \sum_{r=1}^{|C|} \Pr\left(\widetilde{A} = A \middle| A = r\right). \tag{44}$$

Let us continue the proof under the worst scenario that *mvf*(.) tends to select an incorrect class when it breaks ties.

$$\text{RHS of Eq. (44)} \leq 1 - \frac{1}{|C|} \sum_{r=1}^{|C|} \Pr\left(\bigcap_{\substack{u=1 \\ u \neq r}}^{|C|} S_u < S_r \middle| A = r \right) \tag{45}$$

By Eq. (41), we have

$$\text{RHS of Eq. (45)} \leq 2\left(1-\frac{1}{|C|}\sum_{r=1}^{|C|}\Pr\left(\bigcap_{\substack{u=1\\u\neq r}}^{|C|}\widehat{S_u}<\widehat{S_r}\middle|A=r\right)\right) \tag{46}$$

$$\text{RHS of Eq. (46)} = 2\left(1-\frac{1}{|C|}\sum_{r=1}^{|C|}\prod_{\substack{u=1\\u\neq r}}^{|C|}\Pr\left(\widehat{S_u}<\widehat{S_r}\middle|A=r\right)\right) \tag{47}$$

Since $\Pr\left(\widehat{S_u}<\widehat{S_r}\middle|A=r\right)=1-\sum_{\beta=0}^{\infty}\Pr\left(\widehat{S_u}-\widehat{S_r}=\beta\middle|A=r\right)$, Hence, we have:

$$\text{RHS of Eq. (47)} = 2\left(1-\frac{1}{|C|}\sum_{r=1}^{|C|}\prod_{\substack{u=1\\u\neq r}}^{|C|}\left(1-\sum_{\beta=0}^{\infty}\Pr\left(\widehat{S_u}-\widehat{S_r}=\beta\middle|A=r\right)\right)\right) \tag{48}$$

By substituting the $\Pr\left(\widehat{S_u}-\widehat{S_r}=\beta\middle|A=r\right)$ with Skellam PMF, the above equation equals:

$$= 2\left(1-\frac{1}{|C|}\sum_{r=1}^{|C|}\prod_{\substack{u=1\\u\neq r}}^{|C|}\left(1-\sum_{\beta=0}^{\infty}e^{-B(p_{u|r}+p_{r|r})}\times\left(\frac{p_{u|r}}{p_{r|r}}\right)^{\frac{\beta}{2}}\times I_{\beta}(2B\sqrt{p_{u|r}p_{r|r}})\right)\right), \tag{49}$$

where, $I_{\beta(.)}$, is the Bessel function of the first kind of order [8]. □

B.4: Proof of Corollary 2

Proof To prove Corollary 2, we first need to notice the monotonicity of the envelope function of the Bessel function. Specifically, according to [45], the Bessel function $I_{\beta}(x)$ exhibits oscillations without periodicity. As x increases, the amplitude of these oscillations decays asymptotically with $x^{-1/2}$, ultimately approaching 0 when $x\to\infty$. If we denote the upper envelope of $I_{\beta}(x)$ as $env^{upper}I_{\beta}(x)$, then the trend of $env^{upper}I_{\beta}(x)$ also remains positive and decreases monotonically as x increases.

To simplify the proof, we use $env^{upper}I_{\beta}(x)$ to replace $I_{\beta}(x)$ and analyze the trend of the upper bound of error rate P_e in relation to B. We defined the following:

$$\Xi(B)=\sum_{\beta=0}^{\infty}e^{-B\times(p_{u|r}+p_{r|r})}\times\left(\frac{p_{u|r}}{p_{r|r}}\right)^{\frac{\beta}{2}}\times env^{upper}I_{\beta}\left(2B\sqrt{p_{u|r}p_{r|r}}\right), \tag{50}$$

Then Eq. (17) can be rewritten as the following:

$$P_e\leq 2\left(1-\frac{1}{|C|}\sum_{r=1}^{|C|}\prod_{\substack{u=1\\u\neq r}}^{|C|}(1-\Xi(B))\right). \tag{51}$$

From Eq. (49), we observe that $e^{-B\times(p_{u|r}+p_{r|r})}$ is positive and decreases monotonically with B. Since the product of two positive, monotonically decreasing functions is also a monotonically decreasing function, we can conclude that $e^{-B\times(p_{u|r}+p_{r|r})} \times env^{upper} I_\beta\left(2B\sqrt{p_{u|r}p_{r|r}}\right)$ decreases monotonically with B. Consequently, $\Xi(B)$ decreases monotonically with B, $1-\Xi(B)$ increases monotonically with B, and the RHS of Eq. (50) decreases monotonically with B. Therefore, the bound on P_e monotonically decreases with B.

In an extreme case, when $B \to \infty$, then $2B\sqrt{p_{u|r}p_{r|r}} \to \infty$, $env^{upper} I_\beta\left(2B\sqrt{p_{u|r}p_{r|r}}\right) \to 0$, and $e^{-B\times(p_{u|r}+p_{r|r})} \to 0$. So Eq. (49) can be rewritten as:

$$P_e \le 2\left(1-\frac{1}{|C|}\sum_{r=1}^{|C|}\prod_{\substack{u=1\\u\neq r}}^{|C|}\left(1-\sum_{\beta=0}^{\infty}0\times\left(\frac{p_{u|r}}{p_{r|r}}\right)^{\frac{\beta}{2}}\times 0\right)\right) \tag{52}$$

$$\text{RHS of Eq. (51)} = 2\left(1-\frac{1}{|C|}\sum_{r=1}^{|C|}\prod_{\substack{u=1\\u\neq r}}^{|C|}(1-0)\right) = 2\left(1-\frac{|C|}{|C|}\right) = 0. \tag{53}$$

Hence, when $B \to \infty$, we can get $P_e \to 0$. □

B.5: Proof of Corollary 3

Proof To prove Corollary 3, we first need to express the upper bound of the error rate P_e in terms of the generalization error $G_k(w_k)$. We observe that models with smaller $G_k(w_k)$, for a given feature, often exhibit a higher probability of correctly predicting $(\widehat{A_k} = r)$ and a lower probability of incorrectly predicting $(\widehat{A_k} = u, (u \neq r))$, which suggests $G_k(w_k) \propto p_{u|r}$. Hence, we denote $p_{u|r} = \varphi \times G_k(w_k)$, where $\varphi \in \mathbb{R}^+$. Then the upper bound of the error rate P_e of $mvf(.)$ can be rewritten as follows: □

$$P_e \le 2\left(1-\frac{1}{|C|}\sum_{r=1}^{|C|}\prod_{\substack{u=1\\u\neq r}}^{|C|}\left(1-\sum_{\beta=0}^{\infty}e^{-B(\varphi\times G_k(w_k)+p_{r|r})}\left(\frac{\varphi\times G_k(w_k)}{p_{r|r}}\right)^{\frac{\beta}{2}} I_\beta\left(2B\sqrt{\varphi\times G_k(w_k)p_{r|r}}\right)\right)\right), \tag{54}$$

From Eq. (53), we can observe that a smaller $G_k(w_k)$ can contribute to a reduction in the P_e. In the extreme case where $G_k(w_k) = 0$, we have $2B\sqrt{\varphi \times G_k(w_k)\, p_{r|r}} = 0$; $\left(\frac{\varphi\times G_k(w_k)}{p_{r|r}}\right)^{\frac{\beta}{2}} = 0$; $e^{-B\times(\varphi\times G_k(w_k)+p_{r|r})}$ is bounded by 1; for the Bessel function part, we have:

$$I_\beta(x) = I_\beta\left(2B\sqrt{\varphi \times G_k(w_k)\, p_{r|r}}\right) \tag{55}$$

$$= \sum_{t=0}^{\infty}\frac{1}{t!(t+\beta)}\left(\frac{2B\sqrt{\varphi \times G_k(w_k)\, p_{r|r}}}{2}\right)^{2t+\beta} \tag{56}$$

$$= \sum_{t=0}^{\infty}\frac{(-1)^k}{t!(t+\beta)!}(0) = 0. \tag{57}$$

Hence, Eq. (53) can be rewritten as:

$$\mathrm{P_e} \leq 2\left(1-\frac{1}{|C|}\sum_{r=1}^{|C|}\prod_{\substack{u=1\\u\neq r}}^{|C|}\left(1-\sum_{\beta=0}^{\infty}e^{-B\times(\varphi\times G_k(w_k)+p_{r|r})}\times 0\times 0\right)\right) \tag{58}$$

$$\text{RHS of Eq.(57)} = 2\left(1-\frac{1}{|C|}\sum_{r=1}^{|C|}\prod_{\substack{u=1\\u\neq r}}^{|C|}(1-0)\right)=0. \tag{59}$$

Therefore, when $G_k(w_k)\rightarrow 0$, we can get $\mathrm{P_e}\rightarrow 0$.

B.6: Proof of Theorem 3

Proof By Eq. (33), we denote θ_1 as the difference between ρ_k and τ_k, and θ_2 as the difference between $\widetilde{\rho_k}$ and τ_k . Corollary 2 demonstrates that as the number of voters increases, the error rate $\mathrm{P_e}$of the $mvf(.)$ decreases, which implies that the decision made by $mvf(.)$ is more accurate than individual choice. Therefore, after using $mvf(.)$ to correct noisy labels, we have $\theta_1 > \theta_2$.

After plugging this analysis into RHS of Eq. (10), we can get the following:

$$\mathbb{E}_{X_k}\sum_{z=1}^{|C|}\left|\Pr_{\rho_k}\left(Y_k=z\middle|X_k\right)-\Pr_{\tau_k}\left(Y_k=z\middle|X_k\right)\right|>\mathbb{E}_{X_k}\sum_{z=1}^{|C|}\left|\Pr_{\widetilde{\rho_k}}\left(Y_k=z\middle|X_k\right)-\Pr_{\tau_k}\left(Y_k=z\middle|X_k\right)\right| \tag{60}$$

Hence,

$$\Omega\cdot\mathbb{E}_{X_k}\sum_{z=1}^{|C|}\left|\Pr_{\rho_k}\left(Y_k=z\middle|X_k\right)-\Pr_{\tau_k}\left(Y_k=z\middle|X_k\right)\right|>\Omega\cdot\mathbb{E}_{X_k}\sum_{z=1}^{|C|}\left|\Pr_{\widetilde{\rho_k}}\left(Y_k=z\middle|X_k\right)-\Pr_{\tau_k}\left(Y_k=z\middle|X_k\right)\right| \tag{61}$$

Therefore, we can get:

$$G_k\left(w_k^{D_k\sim\rho_k}\right) > G_k\left(w_k^{\widetilde{D_k}\sim\widetilde{\rho_k}}\right). \tag{62}$$

□

B.7: Analysis of communication and computation overhead

Communication overhead analysis. Our method introduces extremely low communication overhead, consistent with traditional DFL methods. In Stage 1, clients train their local models independently, with no additional communication required beyond standard model parameter exchanges. In Stage 2, clients exchange model parameters with their online neighbors, a typical operation in DFL that does not introduce extra communication overhead. In Stage 3, clients fine-tune their local models based on the updated dataset without requiring additional communication. Overall, the communication overhead remains $O(m)$, where m is the number of model parameters exchanged among neighbors during the whole training process.

Computation overhead analysis. The computation overhead of our method is also minimal. In Stage 1, the computational cost is equivalent to traditional DFL, as clients train their local models on their original datasets. In Stage 2, the majority voting process to update labels introduces an $O(n)$ computation overhead, where n is the number of data points. This is a straightforward operation and does not significantly increase the computational

burden. In Stage 3, the extra training epochs for fine-tuning the local models are based on the updated dataset, which is necessary for improving model accuracy. This stage has the same computational cost as the initial training stage and does not introduce additional overhead compared to other existing label correction methods. In summary, the introduced computational overhead is ($O(n)$).

Comparison with other methods. Compared to other label correction methods, our approach has the following advantages: (1) Other methods often require additional data processing and training of auxiliary modules, increasing computational overhead. Our method streamlines this by directly utilizing the results from Stage 1 in subsequent stages. (2) Other methods may require exchanging additional information with a central server during the label correction pre-processing stage, increasing communication overhead. Our method avoids this by not introducing any additional communication overhead, consistent with traditional DFL methods. For instance, in 'FedCorr' by Xu et al., 2022, each iteration of the label correction pre-processing Stage involves all clients calculating the local intrinsic dimensionality (LID) score and per-sample loss for their current local model, which adds computational load. Additionally, the LID score will also be transmitted to the server during each iteration, further contributing to communication overhead. Similarly, 'CLC' by Zeng et al., 2022, mandates clients to calculate a threshold c_t in each training iteration of their label correction pre-processing stage to determine the global threshold c_t^G, again intensifying the computational overhead.

Appendix C: Eperiment supplementery materials

C.1 Network structures

See Tables 10, 11 12 and 13.

C.2: DFL-Compatible adaptations of server-based noisy-label methods

As described in Sect. 5.3, we construct direct DFL-compatible adaptations of FedCorr [50], FedDiv [24], and CLC [55], denoted as FedCorr-DFL, FedDiv-DFL, and CLC-DFL, respectively. In each adaptation, the original server-side

Table 10 MNIST network structure

Layer (Type)	Output shape	Param #
Conv2d-1	[− 1, 10, 24, 24]	260
Conv2d-2	[− 1, 20, 8, 8]	5020
Dropout2d-3	[− 1, 20, 8, 8]	0
Linear-4	[− 1, 50]	16050
Linear-5	[− 1, 10]	510

Table 11 Fashion-MNIST network structure

Layer (Type)	Output Shape	Param #
Conv2d-1	[− 1, 32, 28, 28]	320
Conv2d-2	[− 1, 64, 12, 12]	18496
Dropout2d-3	[− 1, 64, 12, 12]	0
Linear-4	[− 1, 600]	1383000
Linear-5	[− 1, 120]	72120
Linear-6	[− 1, 10]	1210

Table 12 CIFAR-10 network structure

Layer (Type)	Output Shape	Param #
Conv2d-1	[− 1, 64, 16, 16]	1,792
MaxPool2d-2	[− 1, 64, 8, 8]	0
Conv2d-3	[− 1, 192, 8, 8]	110,784
MaxPool2d-4	[− 1, 192, 4, 4]	0
Conv2d-5	[− 1, 384, 4, 4]	663,936
Conv2d-6	[− 1, 256, 4, 4]	884,992
Conv2d-7	[− 1, 256, 4, 4]	590,080
MaxPool2d-8	[− 1, 256, 2, 2]	0
Linear-9	[− 1, 4096]	4,198,400
Linear-10	[− 1, 4096]	16,781,312
Linear-11	[− 1, 10]	40,970

Table 13 Network structure for real-world noisy datasets

Dataset	Structure
CIFAR-10N	ResNet18 (Non-pretrained)
CIFAR-100N	ResNet18 (Non-pretrained)
Clothing1M	ResNet50 (Non-pretrained)
ANIMAL-10N	ResNet18 (Non-pretrained)

operations are performed locally and independently by each client using the information received from its five selected peers.

FedCorr-DFL. The original FedCorr uses cumulative local intrinsic dimensionality (LID) statistics collected by the central server to estimate client-specific noise levels and identify clean and noisy clients. It then fine-tunes a global model using the clients identified as clean and uses the resulting model to correct labels at the clients identified as noisy.

In FedCorr-DFL, each client computes its own cumulative LID statistic and estimated noise level following the original FedCorr procedure. In each communication round, the client exchanges its current model and the required LID-related statistics with its five selected peers. Using its own statistics and those received from the selected peers in the current round, the client applies the original FedCorr client-identification procedure to construct a neighborhood-specific clean/noisy partition over itself and the selected peers in that round. During the clean-client fine-tuning stage, the client aggregates only the current models received from the selected peers that are assigned to the clean subset by its neighborhood-specific identification procedure. If the procedure assigns the client itself to the noisy subset, the client uses the resulting clean-neighborhood model to apply the original FedCorr label-correction procedure to its own local dataset. It then continues decentralized training using the resulting locally corrected dataset.

FedDiv-DFL. The original FedDiv aggregates client task models and local noise-filter parameters at the central server. The aggregated noise-filter parameters form a Federated Noise Filter that represents label-noise information across clients. The global task model and Federated Noise Filter are then used for local noise filtering, pseudo-label generation, predictive-consistency-based sample selection, and local model training.

In FedDiv-DFL, each client constructs its own local noise filter from its per-sample losses following the original FedDiv procedure. In each communication round, the client exchanges its current task model and local noise-filter parameters with its five selected peers. Using its own current noise-filter parameters and those received from the selected peers in the current round, the client applies the original FedDiv filter-aggregation operation to construct a neighborhood-specific noise filter. For task-model aggregation, the client aggregates only the current models received from its five selected peers, following the common DFL aggregation rule. The client then uses the resulting neighborhood model and neighborhood noise filter to perform the original FedDiv noise filtering, pseudo-label generation, predictive-consistency-based sample selection, and local task-model update. All filtering, sample-selection, and pseudo-labeling decisions are made independently by each client and are applied only to its own local training process.

CLC-DFL. The original CLC collects class-wise confidence information and corresponding class counts from participating clients and aggregates them at the central server to construct a system-wide class-wise consensus. This

consensus, together with a global task model, is used to identify samples with potentially incorrect labels and correct them using pseudo-labels.

In CLC-DFL, each client computes its own class-wise confidence information and corresponding class counts following the original CLC procedure. In each communication round, the client exchanges its current task model and the required class-wise information with its five selected peers. Using its own current class-wise information and that received from the selected peers in the current round, the client applies the original CLC consensus-aggregation rule to construct a neighborhood-specific class-wise consensus. For task-model aggregation, the client aggregates only the current models received from its five selected peers, following the common DFL aggregation rule. The client then independently applies the original CLC sample-identification and pseudo-labeling procedures using the resulting neighborhood-specific consensus and neighborhood-model predictions to correct labels in its own local dataset.

Acknowledgements This work is supported in part by the United States National Science Foundation (NSF) under grants CNS-2308761 and CNS-2006998. Any opinions, findings, conclusions, or recommendations expressed in this paper are those of the author(s) and do not necessarily reflect the views of NSF.

Author contributions All authors contributed to the study conception and design. Material preparation, data collection and analysis were performed by Guan Huang. Tao Shu oversees the entire research process of the work. The first draft of the manuscript was written by Guan Huang, and all authors commented on previous versions of the manuscript. All authors read and approved the final manuscript.

Data availability The datasets analysed in this study are publicly available benchmark datasets. MNIST [22] is available through the PyTorch dataset repository (https://pytorch.org/vision/stable/generated/torchvision.datasets.MNIST.html). Fashion-MNIST [48] is available from Zalando Research (https://github.com/zalandoresearch/fashion-mnist). CIFAR-10 [20] is available from the University of Toronto (https://www.cs.toronto.edu/~kriz/cifar.html). CIFAR-10N and CIFAR-100N [46] are available from the UCSC REAL repository (http://ucsc-real.soe.ucsc.edu:1995/Home.html). Clothing1M [49] is available from the authors' repository (https://github.com/Cysu/noisy_label). ANIMAL-10N [38] is publicly available via Kaggle (https://www.kaggle.com/datasets/minseokkim/animal10n-dataset). All datasets were used in accordance with their respective licenses. No new datasets were generated during this study. Synthetic noisy labels and non-IID data partitions were generated following the procedures described in Sect. 5 and are reproducible from the experimental protocol.

Code availability Code will be made publicly available upon acceptance.

Declarations

Competing interests The authors declare no competing interests.

Ethics approval Not applicable.

References

1. Aeeneh S (2023) New bounds on the accuracy of majority voting for multi-class classification. IEEE Trans Neural Netw Learn Syst. https://doi.org/10.1109/TNNLS.2024.3387544
2. Aherne FJ, Thacker NA, Rockett PI (1998) The bhattacharyya metric as an absolute similarity measure for frequency coded data. Kybernetika 34(4):[363]–368. http://eudml.org/doc/33362
3. Assran M, Loizou N, Ballas N (2019) Stochastic gradient push for distributed deep learning. International Conference on Machine Learning. pp 344–353 (**PMLR**)
4. Chang K, Balachandar N, Lam C et al (2018) Distributed deep learning networks among institutions for medical imaging. J Am Med Inform Assoc 25(8):945–954
5. Chen P, Liao BB, Chen G, et al (2019) Understanding and utilizing deep neural networks trained with noisy labels. In: International Conference on Machine Learning, PMLR, pp 1062–1070
6. Chen S, Yu D, Zou Y et al (2022) Decentralized wireless federated learning with differential privacy. IEEE Trans Industr Inf 18(9):6273–6282. https://doi.org/10.1109/TII.2022.3145010
7. Chen Y, Yang X, Qin X, et al (2020) Dealing with Label Quality Disparity in Federated Learning, Springer International Publishing, Cham, pp 108–121. https://doi.org/10.1007/978-3-030-63076-8_8

8. Dobrushkin V (2017) The bessel functions. https://appliedmath.brown.edu/sites/default/files/fractional/35%20TheBesselFunctions.pdf
9. Duan S, Liu C, Cao Z et al (2022) Fed-dr-filter: Using global data representation to reduce the impact of noisy labels on the performance of federated learning. Futur Gener Comput Syst 137:336–348
10. Englesson E, Azizpour H (2024) Robust classification via regression for learning with noisy labels. In: The Twelfth International Conference on Learning Representations, https://openreview.net/forum?id=wfgZc3IMqo
11. Ghosh A, Kumar H, Sastry PS (2017) Robust loss functions under label noise for deep neural networks. Proceedings of the AAAI conference on artificial intelligence
12. Görnitz N, Porbadnigk A, Binder A (2014) Learning and evaluation in presence of non-iid label noise. Artificial Intelligence and Statistics 293–302 (**PMLR**)
13. Hendrycks D, Mazeika M, Wilson D (2018) Using trusted data to train deep networks on labels corrupted by severe noise. Advances in neural information processing systems. p 31
14. Holt K (2021) Mit study finds labelling errors in datasets used to test ai. https://www.engadget.com/mit-datasets-ai-machine-learning-label-errors-040042574.html/
15. Hsu T, Qi H, Brown M (2019) Measuring the effects of non-identical data distribution for federated visual classification. arXiv:1909.06335 arXiv preprint
16. Huang Y, Bert C, Fischer S (2022) Continual learning for peer-to-peer federated learning: A study on automated brain metastasis identification.arXiv:2204.13591 arXiv preprint
17. Kairouz P, McMahan HB, Avent B et al (2021) Advances and open problems in federated learning. Foundations and Trends in Machine Learning 14(1–2):1–210
18. Ke S, C H, X L (2023) Quantifying the impact of label noise on federated learning. AAAI-2023 Workshop on AI for Agriculture and Food Systems https://par.nsf.gov/biblio/10394400
19. Koloskova A, Lin T, Stich SU (2019) Decentralized deep learning with arbitrary communication compression.arXiv:1907.09356 arXiv preprint
20. Krizhevsky A (2009) Learning multiple layers of features from tiny images. Technical Report, University of Toronto, https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf, pages 32–33
21. Lalitha A, Shekhar S, Javidi T (2018) Fully decentralized federated learning. Third workshop on bayesian deep learning (NeurIPS)
22. LeCun Y, Bottou L, Bengio Y et al (1998) Gradient-based learning applied to document recognition. Proc IEEE 86(11):2278–2324
23. Li J, Socher R, Hoi SC (2020) Dividemix: Learning with noisy labels as semi-supervised learning.arXiv:2002.07394 arXiv preprint
24. Li J, Li G, Cheng H, et al (2024) Feddiv: Collaborative noise filtering for federated learning with noisy labels. In: Proceedings of the AAAI Conference on Artificial Intelligence, pp 3118–3126
25. Li X, Huang K, Yang W (2019) On the convergence of fedavg on non-iid data.arXiv:1907.02189 arXiv preprint
26. Li Y, Yang J, Song Y (2017) Learning from noisy labels with distillation. Proceedings of the IEEE international conference on computer vision. pp 1910–1918
27. McMahan B, Moore E, Ramage D, et al (2017) Communication-efficient learning of deep networks from decentralized data. In: Artificial intelligence and statistics, PMLR, pp 1273–1282
28. Mitzenmacher M, Upfal E (2017) Probability and computing: Randomization and probabilistic techniques in algorithms and data analysis. Cambridge University Press, New York, NY, USA
29. Nishi K, Ding Y, Rich A (2021) Augmentation strategies for learning with noisy labels. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp 8022–8031
30. Northcutt C, Jiang L, Chuang I (2021) Confident learning: estimating uncertainty in dataset labels. J Artif Intell Res 70:1373–1411
31. Onoszko N, Karlsson G, Mogren O (2021) Decentralized federated learning of deep neural networks on non-iid data. arXiv:2107.08517 arXiv preprint
32. Patrini G, Rozza A, Menon K, A (2017) Making deep neural networks robust to label noise: A loss correction approach. Proceedings of the IEEE conference on computer vision and pattern recognition. pp 1944–1952
33. Priebe CE, Huang N, Villar S (2022) Deep learning is provably robust to symmetric label noise.arXiv:2210.15083 arXiv preprint
34. Roy AG, Siddiqui S, Pölsterl S (2019) Braintorrent: A peer-to-peer environment for decentralized federated learning. arXiv:1905.06731 arXiv preprint
35. Sheller MJ, Reina GA, Edwards B, et al (2019) Multi-institutional deep learning modeling without sharing patient data: A feasibility study on brain tumor segmentation. In: Brainlesion: Glioma, Multiple Sclerosis, Stroke and Traumatic Brain Injuries: 4th International Workshop, BrainLes 2018, Held in Conjunction with MICCAI 2018, Granada, Spain, September 16, 2018, Revised Selected Papers, Part I 4, Springer, pp 92–104
36. Sheller MJ, Edwards B, Reina GA et al (2020) Federated learning in medicine: facilitating multi-institutional collaborations without sharing patient data. Sci Rep 10(1):12598

37. Shi Y, Zhou Y, Shi Y (2021) Over-the-air decentralized federated learning. In: 2021 IEEE International Symposium on Information Theory (ISIT), IEEE, pp 455–460
38. Song H, Kim M, Lee JG (2019) SELFIE: Refurbishing unclean samples for robust deep learning. In: ICML
39. Tanaka D, Ikami D, Yamasaki T (2018) Joint optimization framework for learning with noisy labels. Proceedings of the IEEE conference on computer vision and pattern recognition. pp 5552–5560
40. Vahdat A (2017) Toward robustness against label noise in training deep discriminative neural networks. Advances in neural information processing systems 30
41. Wang D, Prabhat S, Sambasivan N (2022a) Whose ai dream? in search of the aspiration in data annotation. In: Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems. Association for Computing Machinery, New York, NY, USA, CHI '22,https://doi.org/10.1145/3491102.3502121,
42. Wang J, Sahu AK, Joshi G et al (2022) Matcha: A matching-based link scheduling strategy to speed up distributed optimization. IEEE Trans Signal Process 70:5208–5221
43. Wang Y, Ma X, Chen Z (2019) Symmetric cross entropy for robust learning with noisy labels. Proceedings of the IEEE/CVF international conference on computer vision. pp 322–330
44. Wang Z, Hu G, Hu Q (2020) Training noise-robust deep neural networks via meta-learning. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pp 4524–4533
45. Weber HJ, Arfken GB, Arfken GB (2004) Essential mathematical methods for physicists. Academic Press, Amsterdam, North Holland, https://shop.elsevier.com/books/essential - mathematical-methods- for-physicists-ise / weber/978-0-12-059878-6
46. Wei J, Zhu Z, Cheng H, et al (2022) Learning with noisy labels revisited: A study using real-world human annotations. In: International Conference on Learning Representations, https://openreview.net/forum?id=TBWA6PLJZQm
47. Welinder P, Branson S, Mita T et al (2010) Caltech-UCSD Birds 200. California Institute of Technology, Pasadena, CA, USA
48. Xiao H, Rasul K, Vollgraf R (2017) Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms. arXiv:1708.07747 arXiv preprint
49. Xiao T, Xia T, Yang Y (2015) Learning from massive noisy labeled data for image classification. Proceedings of the IEEE conference on computer vision and pattern recognition. pp 2691–2699
50. Xu J, Chen Z, Quek TQ (2022) Fedcorr: Multi-stage federated learning for label noise correction. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp 10184–10193
51. Yagli S, Dytso A, Poor HV (2020) Information-theoretic bounds on the generalization error and privacy leakage in federated learning. 2020 IEEE 21st International Workshop on Signal Processing Advances in Wireless Communications (SPAWC). IEEE, pp 1–5
52. Yang M, Qian H, Wang X et al (2021) Client selection for federated learning with label noise. IEEE Trans Veh Technol 71(2):2193–2197
53. Yuan L, Ma Y, Su L (2023) Peer-to-peer federated continual learning for naturalistic driving action recognition. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp 5249–5258
54. Yuan L, Wang Z, Sun L et al (2024) Decentralized federated learning: a survey and perspective. IEEE Internet Things J. https://doi.org/10.1109/JIOT.2024.3407584
55. Zeng B, Yang X, Chen Y et al (2022) Clc: A consensus-based label correction approach in federated learning. ACM Trans Intell Syst Technol (TIST) 13(5):1–23

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Authors and Affiliations

Guan Huang[1] · Tao Shu[1]

✉ Tao Shu
tshu@auburn.edu

Guan Huang
gzh0040@auburn.edu

[1] Department of Computer Science and Software Engineering, Auburn University, Auburn, AL 36849, USA